

Manual Tecnico - SimpliCode

Nombre: Angel Emanuel Rodriguez Corado
Carnet: 202404856

SimpliCode IDE

Inicio

Nuevo

Abrir

Guardar

Graficar AST

Opciones ▾

Editor de Texto - SimpliCode

// Escribe o sube un archivo de SimpliCode aqui

Salida por Consola

Aqui aparecera los resultados de ejecucion

Analizar

Guardar Como

Tabla de Tokens (0)

No.	Token	Tipo	Valor	Linea	Columna
Los tokens apareceran aqui despues del analisis					

Errores (0)

No.	Tipo Error	Mensaje	Linea	Columna
No se encontraron errores				

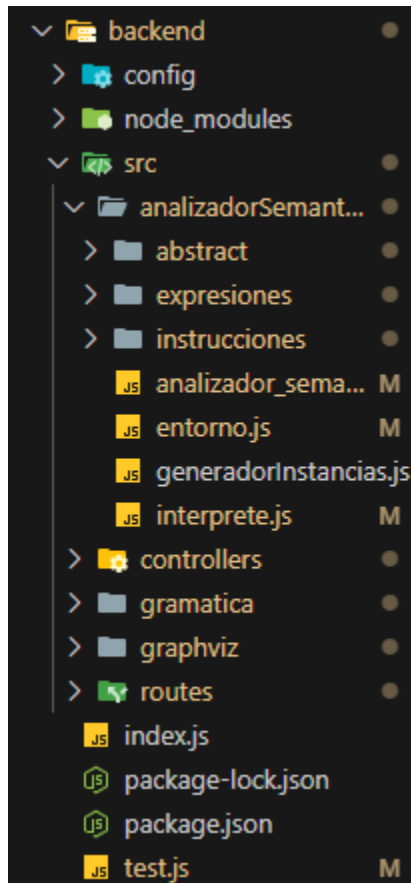
Índice

Índice.....	1
Introduccion.....	2
Backend:.....	3
index.js.....	3
analizador_semantico.js.....	4
entorno.js.....	44
interprete.js.....	51
analyzerController.js.....	52
graphviz_generator.js.....	62
analyzer.js.....	67
Frontend:.....	68
index.js.....	68
ConsoleOutput.jsx.....	68
Editor.jsx.....	69
Header.jsx.....	70
Table.jsx.....	74
App.jsx.....	76

Introduccion

En este manual tecnico se muestra el codigo utilizado para el correcto funcionamiento, analisis y ejecucion del codigo SimpliCode, se muestran los principales metodos, funciones utilizadas para llevar a cabo este proyecto

Backend:



[index.js](#)

```
const express = require('express');
const cors = require('cors');
const analyzerRoutes = require('./src/routes/analyzer');

const app = express();
const PORT = 3000;

// Middlewares
app.use(cors()); // Permite requests desde el frontend
app.use(express.json()); // Para parsear JSON
app.use(express.urlencoded({ extended: true }));

// Rutas
app.use('/api', analyzerRoutes);
```

```
// Ruta de prueba
app.get('/', (req, res) => {
  res.json({
    message: 'Backend de SimpliCode funcionando',
    endpoints: {
      analyze: 'POST /api/analyze'
    }
  });
});

// Manejo de errores
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    message: 'Error interno del servidor'
  });
});

// Ruta no encontrada - CORREGIDO
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: 'Ruta no encontrada'
  });
});

app.listen(PORT, () => {
  console.log(` Servidor corriendo en http://localhost:${PORT} `);
  console.log(` Endpoints disponibles: `);
  console.log(`   POST http://localhost:${PORT}/api/analyze `);
  console.log(`   GET  http://localhost:${PORT}/ `);
});
```

analizador_semantico.js

```
// backend\src\analizadorSemantico\analizador_semantico.js
```

```
const { Entorno, Simbolo } = require('./entorno');
const Interprete = require('./interprete');

class BreakException extends Error {
  constructor() {
    super('Break');
    this.name = 'BreakException';
  }
}

class ContinueException extends Error {
  constructor() {
    super('Continue');
    this.name = 'ContinueException';
  }
}

class ReturnException extends Error {
  constructor(valor) {
    super('Return');
    this.name = 'ReturnException';
    this.valor = valor;
  }
}

class ObjectSymbol extends Simbolo {
  constructor(nombre, atributos = [], metodos = []) {
    super(nombre, 'objeto', { atributos, metodos });
    this.atributos = atributos;
    this.metodos = metodos;
  }

  obtenerAtributo(nombre) {
    return this.atributos.find(attr => attr.name === nombre);
  }

  obtenerMetodo(nombre) {
    return this.metodos.find(method => method.name === nombre);
  }

  agregarMetodo(metodo) {
    this.metodos.push(metodo);
  }
}
```

```
class InstanceSymbol extends Simbolo {
    constructor(nombre, tipoObjeto, valores = {}) {
        super(nombre, 'instancia', valores);
        this.tipoObjeto = tipoObjeto;
        this.valores = valores;
    }

    establecerValor(atributo, valor) {
        this.valores[atributo] = valor;
    }

    obtenerValor(atributo) {
        return this.valores[atributo];
    }
}

class AnalizadorSemantico {
    constructor() {
        this.interprete = new Interprete();
        this.salida = [];
    }

    analizar(ast) {
        try {
            this.salida = [];
            console.log("Iniciando analisis semantico...");

            // evaluar el ast
            this.prepararAST(ast);

            // Ejecutar el programa
            this.interprete.interpretar(ast, (texto) => {
                this.capturarSalida(texto);
            });

            return {
                exito: true,
                mensaje: "Analisis semantico completado",
                salida: this.salida.join('\n')
            };
        } catch (error) {
            // Manejaa ReturnException como terminacion normal en contexto global
        }
    }
}
```

```
    if (error instanceof ReturnException) {
      console.log("Programa terminado por 'retornar'");
      return {
        exito: true,
        mensaje: "Programa terminado por 'retornar'",
        salida: this.salida.join('\n')
      };
    }

    return {
      exito: false,
      error: error.message,
      salida: this.salida.join('\n')
    };
  }
}

prepararAST(ast) {
  if (!ast || !ast.body) {
    throw new Error("AST invalido o vacio");
  }

  // Agregar metodo evaluar al programa principal
  ast.evaluar = (entorno) => {
    for (const instruccion of ast.body) {
      this.evaluarNodo(instruccion, entorno);
    }
  };

  // Preparar todos los nodos hijos recursivamente
  this.prepararNodos(ast.body);
}

prepararNodos(nodos) {
  if (!Array.isArray(nodos)) return;

  for (const nodo of nodos) {
    if (!nodo || typeof nodo !== 'object') {
      continue;
    }

    console.log(`Preparando nodo: ${nodo.type}`);

    switch (nodo.type) {
```

```
case 'Print':
    nodo.evaluar = (entorno) => this.evaluarPrint(nodo, entorno);
    if (nodo.expr) this.prepararNodos([nodo.expr]);
    break;

case 'BinaryOp':
    nodo.evaluar = (entorno) => this.evaluarBinaryOp(nodo, entorno);
    if (nodo.left) this.prepararNodos([nodo.left]);
    if (nodo.right) this.prepararNodos([nodo.right]);
    break;

case 'Literal':
    nodo.evaluar = (entorno) => this.evaluarLiteral(nodo, entorno);
    break;

case 'Identifier':
    nodo.evaluar = (entorno) => this.evaluarIdentifier(nodo, entorno);
    break;

case 'VarDecl':
    nodo.evaluar = (entorno) => this.evaluarVarDecl(nodo, entorno);
    if (nodo.valores && Array.isArray(nodo.valores)) {
        this.prepararNodos(nodo.valores);
    }
    break;

case 'Assign':
    nodo.evaluar = (entorno) => this.evaluarAssign(nodo, entorno);
    if (nodo.target) this.prepararNodos([nodo.target]);
    if (nodo.value) this.prepararNodos([nodo.value]);
    break;

case 'UnaryOp':
    nodo.evaluar = (entorno) => this.evaluarUnaryOp(nodo, entorno);
    if (nodo.expr) this.prepararNodos([nodo.expr]);
    break;

case 'ToLower':
case 'ToUpper':
    nodo.evaluar = (entorno) => this.evaluarFuncionTexto(nodo, entorno);
    if (nodo.expr) this.prepararNodos([nodo.expr]);
    break;

case 'IfStmt':
```

```

    nodo.evaluar = (entorno) => this.evaluarIf(nodo, entorno);
    // Preparar todos los bloques hijos como nodos Block
    if (nodo.condition) this.prepararNodos([nodo.condition]);
    if (nodo.thenBlock) this.prepararNodos([nodo.thenBlock]);
    if (nodo.elseBlock) this.prepararNodos([nodo.elseBlock]);
    if (nodo.elseifs && Array.isArray(nodo.elseifs)) {
        nodo.elseifs.forEach(elseif => {
            if (elseif.condition) this.prepararNodos([elseif.condition]);
            if (elseif.thenBlock) this.prepararNodos([elseif.thenBlock]);
        });
    }
    break;

case 'Block':
    nodo.evaluar = (entorno) => this.evaluarBlock(nodo, entorno);
    if (nodo.statements && Array.isArray(nodo.statements)) {
        this.prepararNodos(nodo.statements);
    }
    break;

case 'Cast':
    nodo.evaluar = (entorno) => this.evaluarCast(nodo, entorno);
    if (nodo.expr) this.prepararNodos([nodo.expr]);
    break;

case 'Increment':
    nodo.evaluar = (entorno) => this.evaluarIncrement(nodo, entorno);
    if (nodo.variable) this.prepararNodos([nodo.variable]);
    break;

case 'VectorDecl':
    nodo.evaluar = (entorno) => this.evaluarVectorDecl(nodo, entorno);
    if (nodo.tamaño) this.prepararNodos([nodo.tamaño]);
    if (nodo.filas && typeof nodo.filas === 'object') this.prepararNodos([nodo.filas]);
    if (nodo.columnas && typeof nodo.columnas === 'object')
this.prepararNodos([nodo.columnas]);
    if (nodo.valores && Array.isArray(nodo.valores)) this.prepararNodos(nodo.valores);
    break;

case 'ArrayAccess':
    nodo.evaluar = (entorno) => this.evaluarArrayAccess(nodo, entorno);
    if (nodo.array) this.prepararNodos([nodo.array]);
    if (nodo.index) this.prepararNodos([nodo.index]);
    if (nodo.index1) this.prepararNodos([nodo.index1]);

```

```
    if (nodo.index2) this.prepararNodos([nodo.index2]);
    break;

case 'WhileStmt':
    nodo.evaluar = (entorno) => this.evaluarWhile(nodo, entorno);
    if (nodo.condition) this.prepararNodos([nodo.condition]);
    if (nodo.body) this.prepararNodos([nodo.body]);
    break;

case 'DoWhileStmt':
    nodo.evaluar = (entorno) => this.evaluarDoWhile(nodo, entorno);
    if (nodo.condition) this.prepararNodos([nodo.condition]);
    if (nodo.body) this.prepararNodos([nodo.body]);
    break;

case 'ForStmt':
    nodo.evaluar = (entorno) => this.evaluarFor(nodo, entorno);
    if (nodo.initialization) this.prepararNodos([nodo.initialization]);
    if (nodo.condition) this.prepararNodos([nodo.condition]);
    if (nodo.update) this.prepararNodos([nodo.update]);
    if (nodo.body) this.prepararNodos([nodo.body]);
    break;

case 'BreakStmt':
    nodo.evaluar = (entorno) => this.evaluarBreak(nodo, entorno);
    break;

case 'ContinueStmt':
    nodo.evaluar = (entorno) => this.evaluarContinue(nodo, entorno);
    break;

case 'ProcedureDecl':
    nodo.evaluar = (entorno) => this.evaluarProcedureDecl(nodo, entorno);
    if (nodo.parameters && Array.isArray(nodo.parameters)) {
        this.prepararNodos(nodo.parameters);
    }
    if (nodo.body) this.prepararNodos([nodo.body]);
    break;

case 'FunctionDecl':
    nodo.evaluar = (entorno) => this.evaluarFunctionDecl(nodo, entorno);
    if (nodo.parameters && Array.isArray(nodo.parameters)) {
        this.prepararNodos(nodo.parameters);
    }
}
```

```
    if (nodo.body) this.prepararNodos([nodo.body]);
    break;

case 'ExecuteCall':
    nodo.evaluar = (entorno) => this.evaluarExecuteCall(nodo, entorno);
    if (nodo.args && Array.isArray(nodo.args)) {
        this.prepararNodos(nodo.args);
    }
    break;

case 'Call':
    nodo.evaluar = (entorno) => this.evaluarCall(nodo, entorno);
    if (nodo.args && Array.isArray(nodo.args)) {
        this.prepararNodos(nodo.args);
    }
    break;

case 'ParamDecl':
    nodo.evaluar = (entorno) => this.evaluarParamDecl(nodo, entorno);
    if (nodo.defaultValue) {
        this.prepararNodos([nodo.defaultValue]);
    }
    break;

case 'Ternary':
    nodo.evaluar = (entorno) => this.evaluarTernary(nodo, entorno);
    if (nodo.cond) this.prepararNodos([nodo.cond]);
    if (nodo.trueExpr) this.prepararNodos([nodo.trueExpr]);
    if (nodo.falseExpr) this.prepararNodos([nodo.falseExpr]);
    break;

case 'ReturnStmt':
    nodo.evaluar = (entorno) => this.evaluarReturnStmt(nodo, entorno);
    if (nodo.value) this.prepararNodos([nodo.value]);
    break;

case 'ObjectDecl':
    nodo.evaluar = (entorno) => this.evaluarObjectDecl(nodo, entorno);
    if (nodo.attributes && Array.isArray(nodo.attributes)) {
        this.prepararNodos(nodo.attributes);
    }
    if (nodo.methods && Array.isArray(nodo.methods)) {
        this.prepararNodos(nodo.methods);
    }
    break;
```

```

    case 'ObjectMethod':
      nodo.evaluar = (entorno) => this.evaluarObjectMethod(nodo, entorno);
      if (nodo.parameters && Array.isArray(nodo.parameters)) {
        this.prepararNodos(nodo.parameters);
      }
      if (nodo.body) this.prepararNodos([nodo.body]);
      break;

    case 'ObjectInstance':
      nodo.evaluar = (entorno) => this.evaluarObjectInstance(nodo, entorno);
      if (nodo.arguments && Array.isArray(nodo.arguments)) {
        this.prepararNodos(nodo.arguments);
      }
      break;

    case 'MethodExecute':
      nodo.evaluar = (entorno) => this.evaluarMethodExecute(nodo, entorno);
      if (nodo.instance) this.prepararNodos([nodo.instance]);
      if (nodo.args && Array.isArray(nodo.args)) {
        this.prepararNodos(nodo.args);
      }
      break;

    case 'AttributeDecl':
      nodo.evaluar = (entorno) => this.evaluarAttributeDecl(nodo, entorno);
      break;

    default:
      console.log(`Tipo de nodo no manejado: ${nodo.type}`);
      nodo.evaluar = () => console.log(`Nodo ${nodo.type} omitido`);
  }
}

//Evaluamos el nodo
evaluarNodo(nodo, entorno) {
  if (nodo && typeof nodo.evaluar === 'function') {
    return nodo.evaluar(entorno);
  }
  console.log(`Nodo sin metodo evaluar:`, nodo);
  return { valor: null, tipo: 'desconocido' };
}

```

```
// Evaluamos el imprimir
evaluarPrint(nodo, entorno) {
  const resultado = this.evaluarNodo(nodo.expr, entorno);
  const textoSalida = String(resultado.valor);

  this.capturarSalida(textoSalida);

  console.log("Print ejecutado:", textoSalida);
  return resultado;
}

//Evaluamos las operaciones binarias
evaluarBinaryOp(nodo, entorno) {
  const izquierda = this.evaluarNodo(nodo.left, entorno);
  const derecha = this.evaluarNodo(nodo.right, entorno);

  console.log(`Operacion: ${izquierda.valor} ${nodo.operator} ${derecha.valor}`);

  let resultado;
  switch (nodo.operator) {
    case '+':
      resultado = izquierda.valor + derecha.valor;
      break;
    case '-':
      resultado = izquierda.valor - derecha.valor;
      break;
    case '*':
      resultado = izquierda.valor * derecha.valor;
      break;
    case '/':
      if (derecha.valor === 0) {
        throw new Error("Division por cero");
      }
      resultado = izquierda.valor / derecha.valor;
      break;
    case '%':
      resultado = izquierda.valor % derecha.valor;
      break;
    case '^':
      resultado = Math.pow(izquierda.valor, derecha.valor);
      break;
    case '>':
      resultado = izquierda.valor > derecha.valor;
      break;
  }
}
```

```

    case '<':
        resultado = izquierda.valor < derecha.valor;
        break;
    case '>=':
        resultado = izquierda.valor >= derecha.valor;
        break;
    case '<=':
        resultado = izquierda.valor <= derecha.valor;
        break;
    case '==':
        resultado = izquierda.valor == derecha.valor;
        break;
    case '!=':
        resultado = izquierda.valor != derecha.valor;
        break;
    case '&&':
        // AND
        resultado = Boolean(izquierda.valor) && Boolean(derecha.valor);
        break;
    case '||':
        // OR
        resultado = Boolean(izquierda.valor) || Boolean(derecha.valor);
        break;
    default:
        throw new Error(`Operador '${nodo.operator}' no implementado`);
}

return {
    valor: resultado,
    tipo: this.obtenerTipoResultado(izquierda.tipo, derecha.tipo, nodo.operator)
};
}

evaluarLiteral(nodo, entorno) {
    console.log(`Literal: ${nodo.value} (${nodo.kind})`);
    return {
        valor: nodo.value,
        tipo: nodo.kind || this.inferirTipo(nodo.value)
    };
}

//Evaluamos los identificadores
evaluarIdentificier(nodo, entorno) {
    const simbolo = entorno.obtener(nodo.name);

```

```
if (!simbolo) {
  throw new Error(`Variable '${nodo.name}' no definida`);
}
console.log(`Identificador: ${nodo.name} = ${simbolo.valor}`);
return {
  valor: simbolo.valor,
  tipo: simbolo.tipo
};
}

evaluarVarDecl(nodo, entorno) {
  console.log(`Declarando variables: ${nodo.ids.join(', ')} como ${nodo.tipo}`);

  const valores = nodo.valores ?
    nodo.valores.map(val => this.evaluarNodo(val, entorno)) :
    Array(nodo.ids.length).fill(this.getValorPorDefecto(nodo.tipo));

  for (let i = 0; i < nodo.ids.length; i++) {
    const id = nodo.ids[i];
    const valor = valores[i] || this.getValorPorDefecto(nodo.tipo);

    entorno.agregar(id, new Simbolo(id, nodo.tipo, valor.valor));
    console.log(`Variable declarada: ${id} = ${valor.valor} (${nodo.tipo})`);
  }

  return { valor: null, tipo: 'void' };
}

//Evaluamos las asignaciones
evaluarAssign(nodo, entorno) {
  const valor = this.evaluarNodo(nodo.value, entorno);

  // Verificar si el target es un acceso a vector
  if (nodo.target.type === 'ArrayAccess') {
    this.evaluarAsignacionVector(nodo.target, valor.valor, entorno);
    console.log(`Elemento de vector asignado`);
    return valor;
  }

  // Asignacion normal a variable
  const nombreVariable = nodo.target.name || nodo.target;
  if (!entorno.actualizar(nombreVariable, valor.valor)) {
    throw new Error(`Variable '${nombreVariable}' no definida para asignacion`);
  }
}
```

```
    console.log(`Variable asignada: ${nombreVariable} = ${valor.valor}`);
    return valor;
}

//Evaluamos las operaciones unarias
evaluarUnaryOp(nodo, entorno) {
    const expr = this.evaluarNodo(nodo.expr, entorno);

    switch (nodo.operator) {
        case '-':
            return { valor: -expr.valor, tipo: expr.tipo };
        case '!':
            if (expr.tipo !== 'booleano') {
                throw new Error("Operador ! requiere operando booleano");
            }
            return { valor: !expr.valor, tipo: 'booleano' };
        default:
            throw new Error(`Operador unario '${nodo.operator}' no implementado`);
    }
}

//Evaluamos las funciones de texto
evaluarFuncionTexto(nodo, entorno) {
    const expr = this.evaluarNodo(nodo.expr, entorno);

    if (expr.tipo !== 'cadena') {
        throw new Error(`Funcion ${nodo.type} requiere cadena, se recibio ${expr.tipo}`);
    }

    let resultado;
    if (nodo.type === 'ToLower') {
        resultado = expr.valor.toLowerCase();
    } else {
        resultado = expr.valor.toUpperCase();
    }

    return { valor: resultado, tipo: 'cadena' };
}

//Evaliamos los si, de lo contrario
evaluarIf(nodo, entorno) {
    console.log(`Evaluando IF...`);
```

```
// Evaluar condicion
const condicion = this.evaluarNodo(nodo.condition, entorno);

if (condicion.tipo !== 'booleano') {
  throw new Error(`La condicion del IF debe ser booleana, no ${condicion.tipo}`);
}

// Ejecutar bloque THEN si condicion es verdadera
if (condicion.valor) {
  console.log(`Condicion verdadera, ejecutando THEN`);
  return this.evaluarNodo(nodo.thenBlock, entorno);
}

// Evaluar ELSE IFs
if (nodo.elseifs && nodo.elseifs.length > 0) {
  for (const elseif of nodo.elseifs) {
    const condElseif = this.evaluarNodo(elseif.condition, entorno);

    if (condElseif.tipo !== 'booleano') {
      throw new Error(`La condicion del O SI debe ser booleana, no ${condElseif.tipo}`);
    }

    if (condElseif.valor) {
      console.log(`Condicion O SI verdadera, ejecutando bloque`);
      return this.evaluarNodo(elseif.thenBlock, entorno);
    }
  }
}

// Ejecutar ELSE si existe
if (nodo.elseBlock) {
  console.log(`Condicion falsa, ejecutando ELSE`);
  return this.evaluarNodo(nodo.elseBlock, entorno);
}

console.log(`Condicion falsa, sin ELSE`);
return { valor: null, tipo: 'void' };
}

//Evaluamos casteos
evaluarCast(nodo, entorno) {
  const expr = this.evaluarNodo(nodo.expr, entorno);

  console.log(`Casteando ${expr.valor} (${expr.tipo}) a ${nodo.targetType}`);
```

```
let valorConvertido;
let tipoResultado = nodo.targetType;

switch (nodo.targetType) {
  case 'entero':
    if (expr.tipo === 'cadena') {
      // Intentar convertir cadena a entero
      const parsed = parseInt(expr.valor);
      if (isNaN(parsed)) {
        throw new Error(`No se puede convertir '${expr.valor}' a entero`);
      }
      valorConvertido = parsed;
    } else if (expr.tipo === 'decimal') {
      valorConvertido = Math.trunc(expr.valor);
    } else if (expr.tipo === 'booleano') {
      valorConvertido = expr.valor ? 1 : 0;
    } else if (expr.tipo === 'caracter') {
      valorConvertido = expr.valor.charCodeAt(0);
    } else {
      valorConvertido = Number(expr.valor);
    }
    break;

  case 'decimal':
    if (expr.tipo === 'cadena') {
      const parsed = parseFloat(expr.valor);
      if (isNaN(parsed)) {
        throw new Error(`No se puede convertir '${expr.valor}' a decimal`);
      }
      valorConvertido = parsed;
    } else {
      valorConvertido = Number(expr.valor);
    }
    break;

  case 'cadena':
    valorConvertido = String(expr.valor);
    break;

  case 'caracter':
    if (expr.tipo === 'entero') {
      // Convertir codigo ASCII a caracter
      if (expr.valor < 0 || expr.valor > 65535) {
```

```

        throw new Error(`Valor ${expr.valor} fuera de rango para caracter`);
    }
    valorConvertido = String.fromCharCode(expr.valor);
} else if (expr.tipo === 'cadena' && expr.valor.length > 0) {
    valorConvertido = expr.valor.charAt(0);
} else {
    throw new Error(`No se puede convertir ${expr.tipo} a caracter`);
}
break;

default:
    throw new Error(`Tipo de casteo no soportado: ${nodo.targetType}`);
}

console.log(`Casteo resultado: ${valorConvertido} (${tipoResultado})`);
return { valor: valorConvertido, tipo: tipoResultado };
}

//Evaluamos incremento ydecremento
evaluarIncrement(nodo, entorno) {
    // Obtener la variable actual
    const nombreVariable = nodo.variable.name;
    const simbolo = entorno.obtener(nombreVariable);

    if (!simbolo) {
        throw new Error(`Variable '${nombreVariable}' no definida para ${nodo.operator}`);
    }

    if (simbolo.tipo !== 'entero' && simbolo.tipo !== 'decimal') {
        throw new Error(`Operador ${nodo.operator} solo aplica a tipos numericos, no
${simbolo.tipo}`);
    }

    const valorActual = simbolo.valor;
    let nuevoValor;

    if (nodo.operator === '++') {
        nuevoValor = valorActual + 1;
    } else if (nodo.operator === '--') {
        nuevoValor = valorActual - 1;
    } else {
        throw new Error(`Operador de incremento no reconocido: ${nodo.operator}`);
    }
}

```

```
// Actualizar la variable
entorno.actualizar(nombreVariable, nuevoValor);

console.log(`Incremento: ${nombreVariable} ${nodo.operator} = ${nuevoValor}`);

// Retornar el valor anterior (comportamiento postfix)
return { valor: valorActual, tipo: simbolo.tipo };
}

//Evaluamos bloques
evaluarBlock(nodo, entorno) {
  console.log(`Ejecutando bloque con ${nodo.statements?.length || 0} sentencias`);

  let ultimoResultado = { valor: null, tipo: 'void' };

  if (nodo.statements && Array.isArray(nodo.statements)) {
    for (const sentencia of nodo.statements) {
      try {
        ultimoResultado = this.evaluarNodo(sentencia, entorno);
      } catch (error) {
        //Propagar ReturnException, BreakException y ContinueException
        if (error instanceof ReturnException ||
            error instanceof BreakException ||
            error instanceof ContinueException) {
          throw error;
        }
        throw error;
      }
    }
  }
}

return ultimoResultado;
}

//Evaluamos mienstras
evaluarWhile(nodo, entorno) {
  console.log(`Iniciando bucle MIENTRAS...`);

  let iteraciones = 0;
  const MAX_ITERACIONES = 1000;

  while (iteraciones < MAX_ITERACIONES) {
    try {
      // Evaluar condicion
```

```
const condicion = this.evaluarNodo(nodo.condition, entorno);

console.log(`Condicion MIENTRAS: ${condicion.valor} (${condicion.tipo})`);

if (condicion.tipo !== 'booleano') {
    throw new Error(`La condicion del MIENTRAS debe ser booleana, no
${condicion.tipo}`);
}

// Salir si condicion es falsa
if (!condicion.valor) {
    console.log(`Condicion falsa, terminando MIENTRAS despues de ${iteraciones}
iteraciones`);
    break;
}

console.log(`Ejecutando iteracion ${iteraciones + 1} del MIENTRAS`);

// Ejecutar cuerpo del bucle
this.evaluarNodo(nodo.body, entorno);

} catch (error) {
    if (error instanceof BreakException) {
        console.log(`DETENER encontrado, saliendo del MIENTRAS`);
        break;
    } else if (error instanceof ContinueException) {
        console.log(`CONTINUAR encontrado, siguiente iteracion del MIENTRAS`);
        // Continuar con la siguiente iteracion
        iteraciones++;
        continue;
    } else if (error instanceof ReturnException) {
        //Propagar ReturnException para salir de la funcion
        console.log(`RETORNAR encontrado en MIENTRAS, propagando...`);
        throw error;
    } else {
        // Relanzar otros errores
        throw error;
    }
}

iteraciones++;

// Prevencion de bucles infinitos
if (iteraciones >= MAX_ITERACIONES) {
```

```
        throw new Error(`Bucle MIENTRAS excedio el limite de ${MAX_ITERACIONES}
iteraciones (posible bucle infinito)`);
    }
}

    console.log(`MIENTRAS completado: ${iteraciones} iteraciones ejecutadas`);
    return { valor: null, tipo: 'void' };
}

// Evaluamos hacer y hasta que
evaluarDoWhile(nodo, entorno) {
    console.log(`Iniciando bucle HACER-HASTA-QUE...`);

    let iteraciones = 0;
    const MAX_ITERACIONES = 1000;
    let condicionResultado;

    do {
        try {
            console.log(`Ejecutando iteracion ${iteraciones + 1} del HACER-HASTA-QUE`);

            // Ejecutar cuerpo del bucle (siempre al menos una vez)
            this.evaluarNodo(nodo.body, entorno);

        } catch (error) {
            if (error instanceof BreakException) {
                console.log(`DETENER encontrado, saliendo del HACER-HASTA-QUE`);
                break;
            } else if (error instanceof ContinueException) {
                console.log(`CONTINUAR encontrado, siguiente iteracion del
HACER-HASTA-QUE`);
                iteraciones++;
                continue;
            } else if (error instanceof ReturnException) {
                //Propagar ReturnException para salir de la funcion
                console.log(`RETORNAR encontrado en HACER-HASTA-QUE, propagando...`);
                throw error;
            } else {
                throw error;
            }
        }
    }

    // Evaluar condicion DESPUES de ejecutar el cuerpo
    condicionResultado = this.evaluarNodo(nodo.condition, entorno);
}
```

```
        console.log(`Condicion HACER-HASTA-QUE: ${condicionResultado.valor}
($condicionResultado.tipo)`);

        if (condicionResultado.tipo !== 'booleano') {
            throw new Error(`La condicion del HACER-HASTA-QUE debe ser booleana, no
${condicionResultado.tipo}`);
        }

        iteraciones++;

        if (iteraciones >= MAX_ITERACIONES) {
            throw new Error(`Bucle HACER-HASTA-QUE excedio el limite de
${MAX_ITERACIONES} iteraciones`);
        }

    } while (condicionResultado.valor); // Continuar mientras la condicion sea verdadera

    console.log(`HACER-HASTA-QUE completado: ${iteraciones} iteraciones ejecutadas`);
    return { valor: null, tipo: 'void' };
}

// Evaluamos para
evaluarFor(nodo, entorno) {
    console.log(`Iniciando bucle PARA...`);

    let iteraciones = 0;
    const MAX_ITERACIONES = 1000;

    try {
        // 1. inicializacion (ejecuta una vez al inicio)
        if (nodo.initialization) {
            console.log(`Ejecutando inicializacion del PARA`);
            this.evaluarNodo(nodo.initialization, entorno);
        }

        // 2. BUCLE PRINCIPAL
        while (iteraciones < MAX_ITERACIONES) {
            try {
                // 2.1 CONDICION (evalua antes de cada iteracion)
                if (nodo.condition) {
                    const condicion = this.evaluarNodo(nodo.condition, entorno);

                    console.log(`Condicion PARA: ${condicion.valor} (${condicion.tipo})`);
```

```
        if (condicion.tipo !== 'booleano') {
            throw new Error(`La condicion del PARA debe ser booleana, no
${condicion.tipo}`);
        }

        // Salir si condicion es falsa
        if (!condicion.valor) {
            console.log(`Condicion falsa, terminando PARA despues de ${iteraciones}
iteraciones`);
            break;
        }
    }

    console.log(`Ejecutando iteracion ${iteraciones + 1} del PARA`);

    // 2.2 CUERPO (ejecuta el cuerpo del bucle)
    this.evaluarNodo(nodo.body, entorno);

} catch (error) {
    if (error instanceof BreakException) {
        console.log(`DETENER encontrado, saliendo del PARA`);
        break;
    } else if (error instanceof ContinueException) {
        console.log(`CONTINUAR encontrado, saltando a actualizacion`);
        // Saltar al paso de actualizacion
    } else if (error instanceof ReturnException) {
        // Propagar ReturnException para salir de la funcion
        console.log(`RETORNAR encontrado en PARA, propagando...`);
        throw error;
    } else {
        throw error;
    }
}

// 2.3 ACTUALIZACION (ejecuta despues de cada iteracion)
if (nodo.update) {
    console.log(`Ejecutando actualizacion del PARA`);
    this.evaluarNodo(nodo.update, entorno);
}

iteraciones++;

if (iteraciones >= MAX_ITERACIONES) {
```

```
        throw new Error(`Bucle PARA excedio el limite de ${MAX_ITERACIONES}
iteraciones`);
    }
}

} catch (error) {
    // Solo capturamos Break/Continue, otros errores se relanzan
    if (!(error instanceof BreakException) && !(error instanceof ContinueException)) {
        throw error;
    }
}

console.log(`PARA completado: ${iteraciones} iteraciones ejecutadas`);
return { valor: null, tipo: 'void' };
}

// Evaluamos detener
evaluarBreak(nodo, entorno) {
    console.log(`Ejecutando sentencia DETENER`);
    throw new BreakException();
}

// Evaluamos continuar
evaluarContinue(nodo, entorno) {
    console.log(`Ejecutando sentencia CONTINUAR`);
    throw new ContinueException();
}

// NEvaluamos procedimientos
evaluarProcedureDecl(nodo, entorno) {
    console.log(`Registrando procedimiento: ${nodo.name}`);

    // Crear simbolo para el procedimiento
    const simboloProcedimiento = new Simbolo(
        nodo.name,
        'procedimiento',
        {
            parametros: nodo.parameters || [],
            body: nodo.body
        }
    );

    // Registrar el procedimiento en el entorno actual
    entorno.agregar(nodo.name, simboloProcedimiento);
}
```

```
    console.log(`Procedimiento '${nodo.name}' registrado con ${nodo.parameters?.length || 0}
parametros`);
    return { valor: null, tipo: 'void' };
}

// Evaluamos funciones
evaluarFunctionDecl(nodo, entorno) {
    console.log(`Registrando funcion: ${nodo.name} -> ${nodo.returnType}`);

    // Crear simbolo para la funcion
    const simboloFuncion = new Simbolo(
        nodo.name,
        'funcion',
        {
            returnType: nodo.returnType,
            parametros: nodo.parameters || [],
            body: nodo.body
        }
    );

    // Registrar la funcion en el entorno actual
    entorno.agregar(nodo.name, simboloFuncion);

    console.log(`Funcion '${nodo.name}' registrada con ${nodo.parameters?.length || 0}
parametros, retorna ${nodo.returnType}`);
    return { valor: null, tipo: 'void' };
}

// Evaluamos llamadas a funciones
evaluarFunctionCall(nodo, entorno) {
    console.log(`Ejecutando funcion: ${nodo.callee}`);

    // Buscar la funcion en el entorno
    const simboloFuncion = entorno.obtener(nodo.callee);

    if (!simboloFuncion || simboloFuncion.tipo !== 'funcion') {
        throw new Error(`Funcion '${nodo.callee}' no definida`);
    }

    const funcion = simboloFuncion.valor;

    // Evaluar argumentos
    const argumentos = nodo.args ?
```

```
nodo.args.map(arg => this.evaluarNodo(arg, entorno)) : [];  
  
// Verificar numero de parametros  
if (argumentos.length !== funcion.parametros.length) {  
    throw new Error(`Numero incorrecto de argumentos para '${nodo.callee}'. Esperados:  
    ${funcion.parametros.length}, Recibidos: ${argumentos.length}`);  
}  
  
// Crear nuevo entorno para la funcion  
const entornoFuncion = new Entorno(entorno);  
entornoFuncion.esFuncion = true; // Marcar como contexto de funcion  
  
// Registrar parametros en el nuevo entorno  
for (let i = 0; i < funcion.parametros.length; i++) {  
    const parametro = funcion.parametros[i];  
    const argumento = argumentos[i];  
  
    // Verificar tipos de parametros  
    if (parametro.tipo !== argumento.tipo) {  
        // Permitir conversion implicita de entero a decimal  
        if (parametro.tipo === 'decimal' && argumento.tipo === 'entero') {  
            console.log(`Conversion implicita: ${argumento.valor} (entero) ->  
            ${argumento.valor}.0 (decimal)`);  
        } else {  
            throw new Error(`Tipo incorrecto para parametro '${parametro.name}'. Esperado:  
            ${parametro.tipo}, Recibido: ${argumento.tipo}`);  
        }  
    }  
  
    entornoFuncion.agregar(  
        parametro.name,  
        new Simbolo(  
            parametro.name,  
            parametro.tipo,  
            argumento.valor  
        )  
    );  
  
    console.log(`Parametro '${parametro.name}' = ${argumento.valor} (${parametro.tipo})`);  
}  
  
// Configurar capturador de salida para la funcion  
entornoFuncion.setCapturadorSalida((texto) => {  
    this.capturarSalida(texto);
```

```

});

// Ejecutar el cuerpo de la funcion con manejo de retorno
console.log(`Ejecutando cuerpo de la funcion '${nodo.callee}'`);
try {
    const resultado = this.evaluarNodo(funcion.body, entornoFuncion);

    // Si llegamos aqui, no hubo retorno explicito
    if (funcion.returnType !== 'void') {
        throw new Error(`Funcion '${nodo.callee}' debe retornar un valor de tipo
${funcion.returnType}`);
    }

    console.log(`Funcion '${nodo.callee}' ejecutada sin retorno explicito`);
    return { valor: null, tipo: 'void' };

} catch (error) {
    if (error instanceof ReturnException) {
        // Verificar que el tipo de retorno coincida
        if (funcion.returnType !== 'void' && error.valor.tipo !== funcion.returnType) {
            // Permitir conversion implicita de entero a decimal
            if (!(funcion.returnType === 'decimal' && error.valor.tipo === 'entero')) {
                throw new Error(`Tipo de retorno incorrecto en '${nodo.callee}'. Esperado:
${funcion.returnType}, Obtenido: ${error.valor.tipo}`);
            }
        }

        console.log(`Funcion '${nodo.callee}' retorno: ${error.valor.valor} (${error.valor.tipo})`);
        return error.valor;

    } else if (error instanceof BreakException || error instanceof ContinueException) {
        throw new Error(`${error.name}' no permitido fuera de ciclos`);
    } else {
        throw error;
    }
}

// Evaluamos llamadas a procedimientos
evaluarExecuteCall(nodo, entorno) {
    console.log(`Ejecutando procedimiento con EJECUTAR: ${nodo.callee}`);

    // Buscar el procedimiento en el entorno
    const simboloProcedimiento = entorno.obtener(nodo.callee);

```

```
if (!simboloProcedimiento || simboloProcedimiento.tipo !== 'procedimiento') {
  throw new Error(`Procedimiento '${nodo.callee}' no definido`);
}

const procedimiento = simboloProcedimiento.valor;

// Evaluar argumentos
const argumentos = nodo.args ?
  nodo.args.map(arg => this.evaluarNodo(arg, entorno)) : [];

// Verificar numero de parametros
if (argumentos.length !== procedimiento.parametros.length) {
  throw new Error(`Numero incorrecto de argumentos para '${nodo.callee}'. Esperados:
${procedimiento.parametros.length}, Recibidos: ${argumentos.length}`);
}

// Crear nuevo entorno para el procedimiento
const entornoProcedimiento = new Entorno(entorno);
entornoProcedimiento.esFuncion = true; // Marcar como contexto de funcion/procedimiento

// Registrar parametros en el nuevo entorno
for (let i = 0; i < procedimiento.parametros.length; i++) {
  const parametro = procedimiento.parametros[i];
  const argumento = argumentos[i];

  // Verificar tipos de parametros
  if (parametro.tipo !== argumento.tipo) {
    // Permitir conversion implicita de entero a decimal
    if (parametro.tipo === 'decimal' && argumento.tipo === 'entero') {
      console.log(`Conversion implicita: ${argumento.valor} (entero) ->
${argumento.valor}.0 (decimal)`);
    } else {
      throw new Error(`Tipo incorrecto para parametro '${parametro.name}'. Esperado:
${parametro.tipo}, Recibido: ${argumento.tipo}`);
    }
  }

  entornoProcedimiento.agregar(
    parametro.name,
    new Simbolo(
      parametro.name,
      parametro.tipo,
      argumento.valor
    )
  )
}
```

```

    )
  );

  console.log(`Parametro '${parametro.name}' = ${argumento.valor} (${parametro.tipo})`);
}

// Configurar capturador de salida para el procedimiento
entornoProcedimiento.setCapturadorSalida((texto) => {
  this.capturarSalida(texto);
});

// Ejecutar el cuerpo del procedimiento
console.log(`Ejecutando cuerpo del procedimiento '${nodo.callee}'`);
try {
  const resultado = this.evaluarNodo(procedimiento.body, entornoProcedimiento);
  console.log(`Procedimiento '${nodo.callee}' ejecutado exitosamente`);
  return { valor: null, tipo: 'void' }; // Los procedimientos no retornan valor
} catch (error) {
  if (error instanceof ReturnException) {
    // Procedimientos pueden usar retornar sin valor
    console.log(`Procedimiento '${nodo.callee}' terminado por retornar`);
    return { valor: null, tipo: 'void' };
  } else if (error instanceof BreakException || error instanceof ContinueException) {
    throw new Error(`${error.name} no permitido fuera de ciclos`);
  }
  throw error;
}
}

// Evaluamos llamadas (funciones o procedimientos)
evaluarCall(nodo, entorno) {
  console.log(`Llamando: ${nodo.callee}`);

  // Buscar el simbolo (puede ser procedimiento o funcion)
  const simbolo = entorno.obtener(nodo.callee);

  if (!simbolo) {
    throw new Error(`${nodo.callee} no esta definido`);
  }

  if (simbolo.tipo === 'procedimiento') {
    return this.evaluarExecuteCall({
      callee: nodo.callee,
      args: nodo.args,
    });
  }
}

```

```
        type: 'ExecuteCall'
      }, entorno);
    } else if (simbolo.tipo === 'funcion') {
      return this.evaluarFunctionCall({
        callee: nodo.callee,
        args: nodo.args,
        type: 'Call'
      }, entorno);
    } else {
      throw new Error(`${nodo.callee} no es un procedimiento o funcion`);
    }
  }
}

// Evaluamos declaracion de parametros
evaluarParamDecl(nodo, entorno) {
  return {
    valor: nodo.name,
    tipo: nodo.tipo,
    defaultValue: nodo.defaultValue
  };
}

// Utilidades de tipo
obtenerTipoResultado(tipoIzq, tipoDer, operador) {
  const operadoresComparacion = ['>', '<', '>=', '<=', '==', '!='];
  if (operadoresComparacion.includes(operador)) {
    return 'booleano';
  }

  if (operador === '&&' || operador === '||') {
    return 'booleano';
  }

  if (operador === '+') {
    if (tipoIzq === 'cadena' || tipoDer === 'cadena') {
      return 'cadena';
    }
  }

  if (tipoIzq === 'decimal' || tipoDer === 'decimal') {
    return 'decimal';
  }

  return 'entero';
}
```

```
inferirTipo(valor) {
  if (typeof valor === 'number') {
    return Number.isInteger(valor) ? 'entero' : 'decimal';
  } else if (typeof valor === 'string') {
    return 'cadena';
  } else if (typeof valor === 'boolean') {
    return 'booleano';
  }
  return 'desconocido';
}

getValorPorDefecto(tipo) {
  switch (tipo) {
    case 'entero': return { valor: 0, tipo: 'entero' };
    case 'decimal': return { valor: 0.0, tipo: 'decimal' };
    case 'booleano': return { valor: false, tipo: 'booleano' };
    case 'caracter': return { valor: '\0', tipo: 'caracter' };
    case 'cadena': return { valor: "", tipo: 'cadena' };
    default: return { valor: null, tipo: 'desconocido' };
  }
}

capturarSalida(texto) {
  const textoLimpio = texto.toString().trim();

  if (textoLimpio && (this.salida.length === 0 || this.salida[this.salida.length - 1] !==
textoLimpio)) {
    this.salida.push(textoLimpio);
    console.log("Salida capturada:", textoLimpio);
  }
}

obtenerSalida() {
  return this.salida.join("\n");
}

limpiarSalida() {
  this.salida = [];
}

// Evaluamos declaracion de vectores
evaluarVectorDecl(nodo, entorno) {
  console.log(`Declarando vector: ${nodo.id} como ${nodo.tipo}`);
```

```

const simbolo = new Simbolo(nodo.id, nodo.tipo, null);

try {
  if (nodo.dimensiones === 1) {
    if (nodo.tamaño) {
      // Vector con tamaño específico: vector entero[4]
      const tamaño = this.evaluarNodo(nodo.tamaño, entorno);
      if (tamaño.tipo !== 'entero') {
        throw new Error(`El tamaño del vector debe ser entero, no ${tamaño.tipo}`);
      }
      const tipoElemento = nodo.tipoElemento || nodo.tipo.replace('[', '');
      simbolo.configurarVector(1, [tamaño.valor], tipoElemento);
      console.log(`Vector 1D creado: ${nodo.id}[${tamaño.valor}]`);
    } else if (nodo.valores && Array.isArray(nodo.valores)) {
      // Vector con valores literales: ["Hola", "Mundo"]
      const valores = nodo.valores.map(val => this.evaluarNodo(val, entorno));
      const valoresSimples = valores.map(v => v.valor);
      const tipoElemento = this.inferirTipoDeValores(valoresSimples);
      simbolo.configurarVectorConValores(valoresSimples, tipoElemento);
      console.log(`Vector 1D con literales: ${nodo.id} = [${valoresSimples.join(', ')}]`);
    }
  } else if (nodo.dimensiones === 2) {
    if (nodo.filas && nodo.columnas && typeof nodo.filas === 'object' && typeof
nodo.columnas === 'object') {
      // Vector 2D con tamaños: vector caracter[2][3]
      const filas = this.evaluarNodo(nodo.filas, entorno);
      const columnas = this.evaluarNodo(nodo.columnas, entorno);

      if (filas.tipo !== 'entero' || columnas.tipo !== 'entero') {
        throw new Error(`Los tamaños del vector deben ser enteros`);
      }
      const tipoElemento = nodo.tipoElemento || nodo.tipo.replace('[[]', '');
      simbolo.configurarVector(2, [filas.valor, columnas.valor], tipoElemento);
      console.log(`Vector 2D creado: ${nodo.id}[${filas.valor}][${columnas.valor}]`);
    } else if (nodo.valores === "[" || !Array.isArray(nodo.valores)) {
      console.log(`Parser no proceso el literal 2D, creando matriz del ejemplo: [[1, 2], [3,
4]]`);

      // Crear EXACTAMENTE la matriz del código fuente
      const tipoElemento = nodo.tipo.replace('[[]', '');
      const matrizEjemplo = [
        [1, 2],
        [3, 4]
      ];
    }
  }
}

```

```

        simbolo.configurarVectorConValores(matrizEjemplo, tipoElemento);
    } else {
        // Procesamiento normal cuando el parser funciona
        const matriz = this.evaluarArrayLiteral2D(nodo.valores, entorno);
        const tipoElemento = this.inferirTipoDeMatriz(matriz);
        simbolo.configurarVectorConValores(matriz, tipoElemento);
        console.log(`Vector 2D con literales: ${nodo.id} = matriz
${matriz.length}x${matriz[0]?.length || 0}`);
    }
}

entorno.agregar(nodo.id, simbolo);
console.log(`Vector declarado exitosamente: ${nodo.id} con ${simbolo.dimensiones}
dimensiones`);

return { valor: simbolo.valor, tipo: nodo.tipo };

} catch (error) {
    console.error(`Error declarando vector ${nodo.id}:`, error.message);
    // Crear vector con valores por defecto como fallback
    const tipoBase = nodo.tipo.replace('[ ]', '').replace('[', '');
    if (nodo.dimensiones === 1) {
        simbolo.configurarVector(1, [5], tipoBase);
    } else {
        simbolo.configurarVector(2, [2, 2], tipoBase);
    }
    entorno.agregar(nodo.id, simbolo);
    console.log(`Vector creado con valores por defecto: ${nodo.id}`);
    return { valor: simbolo.valor, tipo: nodo.tipo };
}
}

// Evaluamos acceso a vectores
evaluarArrayAccess(nodo, entorno) {
    console.log(`Evaluando acceso a array: ${nodo.type}`);

    // Caso especial: acceso anidado para vectores 2D
    if (nodo.array.type === 'ArrayAccess') {
        // Este es el caso: matriz2[1][0] - dos ArrayAccess anidados
        return this.evaluarArrayAccessAnidado(nodo, entorno);
    }

    // Caso normal: acceso simple vector[indice]
    const nombreArray = nodo.array.name;

```

```
const simbolo = entorno.obtener(nombreArray);

if (!simbolo || !simbolo.esVector) {
  throw new Error(`${nombreArray}' no es un vector o no esta definido`);
}

const indice = this.evaluarNodo(nodo.index, entorno);
if (indice.tipo !== 'entero') {
  throw new Error('El indice del vector debe ser entero, no ${indice.tipo}');
}

const indices = [indice.valor];
const elemento = simbolo.obtenerElemento(indices);
const tipoElemento = simbolo.tipoElemento;

console.log(`Acceso a vector: ${nombreArray}[${indices.join(',')}] = ${elemento}`);

return { valor: elemento, tipo: tipoElemento };
}

evaluarArrayAccessAnidado(nodo, entorno) {
  console.log(`Manejando acceso anidado a vector 2D`);

  // Obtener el primer acceso: matriz2[1]
  const primerAccess = nodo.array;
  const nombreArray = primerAccess.array.name;
  const simbolo = entorno.obtener(nombreArray);

  if (!simbolo || !simbolo.esVector) {
    throw new Error(`${nombreArray}' no es un vector o no esta definido`);
  }

  if (simbolo.dimensiones !== 2) {
    throw new Error(`${nombreArray}' no es un vector 2D`);
  }

  // Evaluar ambos indices
  const indice1 = this.evaluarNodo(primerAccess.index, entorno);
  const indice2 = this.evaluarNodo(nodo.index, entorno);

  if (indice1.tipo !== 'entero' || indice2.tipo !== 'entero') {
    throw new Error('Los indices del vector deben ser enteros');
  }
}
```

```

    const indices = [indice1.valor, indice2.valor];
    const elemento = simbolo.obtenerElemento(indices);
    const tipoElemento = simbolo.tipoElemento;

    console.log(`Acceso a vector 2D: ${nombreArray}[${indices.join(',')}] = ${elemento}`);

    return { valor: elemento, tipo: tipoElemento };
  }

  // Evaluamos literales de matrices 2D
  evaluarArrayLiteral2D(listaFilas, entorno) {
    console.log('Evaluando literal de matriz 2D...');

    const matriz = [];

    // Verificar si listaFilas es un array de arrays
    if (Array.isArray(listaFilas)) {
      for (const fila of listaFilas) {
        if (Array.isArray(fila)) {
          const filaEvaluada = [];
          for (const elemento of fila) {
            if (elemento && typeof elemento === 'object') {
              const resultado = this.evaluarNodo(elemento, entorno);
              filaEvaluada.push(resultado.valor);
            } else {
              // Si es un valor directo (no nodo)
              filaEvaluada.push(elemento);
            }
          }
          matriz.push(filaEvaluada);
        }
      }
    }

    console.log(`Matriz 2D evaluada: ${matriz.length} filas`);
    return matriz;
  }

  // Evaluamos asignacion a vectores
  evaluarAsignacionVector(nodoAccess, valor, entorno) {
    // Caso especial: asignacion anidada para vectores 2D
    if (nodoAccess.array.type === 'ArrayAccess') {
      return this.evaluarAsignacionVectorAnidada(nodoAccess, valor, entorno);
    }
  }

```

```
// Caso normal: asignacion simple vector[indice] = valor
const nombreArray = nodoAccess.array.name;
const simbolo = entorno.obtener(nombreArray);

if (!simbolo || !simbolo.esVector) {
  throw new Error(`${nombreArray}' no es un vector o no esta definido`);
}

const indice = this.evaluarNodo(nodoAccess.index, entorno);
if (indice.tipo !== 'entero') {
  throw new Error(`El indice del vector debe ser entero, no ${indice.tipo}`);
}

const indices = [indice.valor];
simbolo.asignarElemento(indices, valor);
console.log(`Vector asignado: ${nombreArray}[${indices.join(',')}] = ${valor}`);
}

evaluarAsignacionVectorAnidada(nodoAccess, valor, entorno) {
  console.log(`Manejando asignacion anidada a vector 2D`);

  // Obtener el primer acceso: matriz[0][1] = valor
  const primerAccess = nodoAccess.array;
  const nombreArray = primerAccess.array.name;
  const simbolo = entorno.obtener(nombreArray);

  if (!simbolo || !simbolo.esVector) {
    throw new Error(`${nombreArray}' no es un vector o no esta definido`);
  }

  if (simbolo.dimensiones !== 2) {
    throw new Error(`${nombreArray}' no es un vector 2D`);
  }

  // Evaluar ambos indices
  const indice1 = this.evaluarNodo(primerAccess.index, entorno);
  const indice2 = this.evaluarNodo(nodoAccess.index, entorno);

  if (indice1.tipo !== 'entero' || indice2.tipo !== 'entero') {
    throw new Error(`Los indices del vector deben ser enteros`);
  }

  const indices = [indice1.valor, indice2.valor];
```

```

    simbolo.asignarElemento(indices, valor);
    console.log(`Vector 2D asignado: ${nombreArray}[${indices.join(',')}] = ${valor}`);
}

//Metodo para inferir tipo de valores en array
inferirTipoDeValores(valores) {
    if (valores.length === 0) return 'entero';

    const primerValor = valores[0];
    if (typeof primerValor === 'string') return 'cadena';
    if (typeof primerValor === 'number') {
        return Number.isInteger(primerValor) ? 'entero' : 'decimal';
    }
    if (typeof primerValor === 'boolean') return 'booleano';
    return 'entero';
}

// Metodo para inferir tipo de matriz
inferirTipoDeMatriz(matriz) {
    if (matriz.length === 0 || matriz[0].length === 0) return 'entero';

    const primerElemento = matriz[0][0];
    if (typeof primerElemento === 'string') return 'cadena';
    if (typeof primerElemento === 'number') {
        return Number.isInteger(primerElemento) ? 'entero' : 'decimal';
    }
    if (typeof primerElemento === 'boolean') return 'booleano';
    return 'entero';
}

// Evaluamos operador ternario
evaluarTernary(nodo, entorno) {
    console.log(`Evaluando operador ternario`);

    // Evaluar la condicion
    const condicion = this.evaluarNodo(nodo.cond, entorno);

    if (condicion.tipo !== 'booleano') {
        throw new Error(`La condicion del operador ternario debe ser booleana, no ${condicion.tipo}`);
    }

    // Evaluar la rama correspondiente
    let resultado;

```

```
if (condicion.valor) {
  console.log(`Condicion verdadera, ejecutando rama true`);
  resultado = this.evaluarNodo(nodo.trueExpr, entorno);
} else {
  console.log(`Condicion falsa, ejecutando rama false`);
  resultado = this.evaluarNodo(nodo.falseExpr, entorno);
}

console.log(`Ternario resultado: ${resultado.valor} (${resultado.tipo})`);
return resultado;
}

// Evaluamos retornar
evaluarReturnStmt(nodo, entorno) {
  console.log(`Ejecutando retornar`);

  let valorRetorno = { valor: null, tipo: 'void' };

  if (nodo.value) {
    valorRetorno = this.evaluarNodo(nodo.value, entorno);
    console.log(`Retornando: ${valorRetorno.valor} (${valorRetorno.tipo})`);
  } else {
    console.log(`Retornando sin valor`);
  }

  // Lanzar excepcion para interrumpir el flujo
  throw new ReturnException(valorRetorno);
}

// Evaluamos declaracion de objetos
evaluarObjectDecl(nodo, entorno) {
  console.log(`Declarando objeto: ${nodo.name}`);

  const atributos = nodo.attributes || [];
  const metodos = nodo.methods || [];

  console.log(`Atributos: ${atributos.length}`);
  console.log(`Metodos: ${metodos.length}`);

  // Crear simbolo para el objeto
  const simboloObjeto = new ObjectSymbol(nodo.name, atributos, metodos);

  // Registrar el objeto en el entorno
  entorno.agregar(nodo.name, simboloObjeto);
}
```

```
    console.log(` Objeto '${nodo.name}' registrado exitosamente`);
    return { valor: null, tipo: 'void' };
}

// Evaluamos metodo de objetos
evaluarObjectMethod(nodo, entorno) {
    console.log(` Registrando metodo '${nodo.methodName}' para objeto
'${nodo.objectName}'`);

    // Buscar el objeto en el entorno
    const simboloObjeto = entorno.obtener(nodo.objectName);

    if (!simboloObjeto || simboloObjeto.tipo !== 'objeto') {
        throw new Error(` Objeto '${nodo.objectName}' no definido para el metodo
'${nodo.methodName}'`);
    }

    // Crear el metodo
    const metodo = {
        name: nodo.methodName,
        parameters: nodo.parameters || [],
        body: nodo.body
    };

    // Agregar el metodo al objeto
    simboloObjeto.agregarMetodo(metodo);

    console.log(` Metodo '${nodo.methodName}' registrado para objeto
'${nodo.objectName}'`);
    return { valor: null, tipo: 'void' };
}

// Evaluamos instancia de objetos
evaluarObjectInstance(nodo, entorno) {
    console.log(` Creando instancia de objeto: ${nodo.instanceName} -> ${nodo.objectType}`);

    // Buscar la definicion del objeto
    const simboloObjeto = entorno.obtener(nodo.objectType);

    if (!simboloObjeto || simboloObjeto.tipo !== 'objeto') {
        throw new Error(` Tipo de objeto '${nodo.objectType}' no definido`);
    }
}
```

```
// Verificar que el constructor coincida
if (nodo.constructor !== nodo.objectType) {
  throw new Error(`Constructor '${nodo.constructor}' no coincide con el tipo de objeto
'${nodo.objectType}'`);
}

// Evaluar argumentos
const argumentos = nodo.arguments ?
  nodo.arguments.map(arg => this.evaluarNodo(arg, entorno)) : [];

// Verificar numero de argumentos
if (argumentos.length !== simboloObjeto atributos.length) {
  throw new Error(`Numero incorrecto de argumentos para '${nodo.objectType}'.
Esperados: ${simboloObjeto atributos.length}, Recibidos: ${argumentos.length}`);
}

// Crear instancia con valores iniciales
const valoresIniciales = {};
simboloObjeto atributos.forEach((atributo, index) => {
  const argumento = argumentos[index];

  // Verificar tipos
  if (atributo.tipo !== argumento.tipo) {
    throw new Error(`Tipo incorrecto para atributo '${atributo.name}'. Esperado:
${atributo.tipo}, Recibido: ${argumento.tipo}`);
  }

  valoresIniciales[atributo.name] = argumento.valor;
  console.log(` ${atributo.name} = ${argumento.valor} (${atributo.tipo})`);
});

// Crear simbolo para la instancia
const simboloInstancia = new InstanceSymbol(nodo.instanceName, nodo.objectType,
valoresIniciales);

// Registrar la instancia en el entorno
entorno.agregar(nodo.instanceName, simboloInstancia);

console.log(` Instancia '${nodo.instanceName}' creada exitosamente`);
return { valor: valoresIniciales, tipo: 'instancia' };
}

// Evaluamos ejecucion de metodos en instancias
evaluarMethodExecute(nodo, entorno) {
```

```
console.log(` Ejecutando metodo: ${nodo.instance.name}.${nodo.method}`);

// Obtener la instancia
const simboloInstancia = entorno.obtener(nodo.instance.name);

if (!simboloInstancia || simboloInstancia.tipo !== 'instancia') {
  throw new Error(`Instancia '${nodo.instance.name}' no definida`);
}

// Obtener la definicion del objeto
const simboloObjeto = entorno.obtener(simboloInstancia.tipoObjeto);

if (!simboloObjeto || simboloObjeto.tipo !== 'objeto') {
  throw new Error(`Tipo de objeto '${simboloInstancia.tipoObjeto}' no definido`);
}

// Buscar el metodo
const metodo = simboloObjeto.obtenerMetodo(nodo.method);

if (!metodo) {
  throw new Error(`Metodo '${nodo.method}' no definido para objeto
'${simboloInstancia.tipoObjeto}'`);
}

// Evaluar argumentos
const argumentos = nodo.args ?
  nodo.args.map(arg => this.evaluarNodo(arg, entorno)) : [];

// Verificar numero de parametros
if (argumentos.length !== metodo.parameters.length) {
  throw new Error(`Numero incorrecto de argumentos para '${nodo.method}'. Esperados:
${metodo.parameters.length}, Recibidos: ${argumentos.length}`);
}

// Crear entorno para el metodo
const entornoMetodo = new Entorno(entorno);
entornoMetodo.esMetodo = true;

// Agregar referencia a 'this' (la instancia actual)
entornoMetodo.agregar('this', simboloInstancia);

// Registrar parametros en el nuevo entorno
for (let i = 0; i < metodo.parameters.length; i++) {
  const parametro = metodo.parameters[i];
```

```
const argumento = argumentos[i];

// Verificar tipos de parametros
if (parametro.tipo !== argumento.tipo) {
  throw new Error(`Tipo incorrecto para parametro '${parametro.name}'. Esperado:
${parametro.tipo}, Recibido: ${argumento.tipo}`);
}

entornoMetodo.agregar(
  parametro.name,
  new Simbolo(
    parametro.name,
    parametro.tipo,
    argumento.valor
  )
);

console.log(` Parametro '${parametro.name}' = ${argumento.valor}
(${parametro.tipo})`);
}

// Tambien agregar los atributos de la instancia al entorno del metodo
Object.keys(simboloInstancia.valores).forEach(atributo => {
  entornoMetodo.agregar(
    atributo,
    new Simbolo(
      atributo,
      this.obtenerTipoAtributo(simboloObjeto, atributo),
      simboloInstancia.valores[atributo]
    )
  );
});

// Configurar capturador de salida para el metodo
entornoMetodo.setCapturadorSalida((texto) => {
  this.capturarSalida(texto);
});

// Ejecutar el cuerpo del metodo
console.log(` Ejecutando cuerpo del metodo '${nodo.method}'`);
try {
  const resultado = this.evaluarNodo(metodo.body, entornoMetodo);
  console.log(` Metodo '${nodo.method}' ejecutado exitosamente`);
  return resultado || { valor: null, tipo: 'void' };
}
```

```

    } catch (error) {
      if (error instanceof ReturnException) {
        console.log(` Metodo '${nodo.method}' retorno: ${error.valor.valor}`);
        return error.valor;
      } else if (error instanceof BreakException || error instanceof ContinueException) {
        throw new Error(`${error.name}' no permitido en metodos de objeto`);
      } else {
        throw error;
      }
    }
  }
}

evaluarAttributeDecl(nodo, entorno) {
  // Los atributos se evaluan cuando se crea la instancia del objeto
  console.log(` Atributo: ${nodo.name} (${nodo.tipo})`);
  return {
    valor: nodo.name,
    tipo: 'atributo',
    name: nodo.name,
    tipoAtributo: nodo.tipo
  };
}

// Funcion auxiliar para obtener el tipo de un atributo
obtenerTipoAtributo(simboloObjeto, nombreAtributo) {
  const atributo = simboloObjeto.atributos.find(attr => attr.name === nombreAtributo);
  return atributo ? atributo.tipo : 'desconocido';
}
}

module.exports = AnalizadorSemantico;

```

[entorno.js](#)

```

class Entorno {
  constructor(padre = null) {
    this.padre = padre;
    this.tabla = new Map();
    this.capturarSalida = null;
    this.esFuncion = false;
  }
}

```

```
        this.esMetodo = false;
    }

    agregar(nombre, simbolo) {
        this.tabla.set(nombre, simbolo);
    }

    obtener(nombre) {
        let entornoActual = this;
        while (entornoActual !== null) {
            if (entornoActual.tabla.has(nombre)) {
                return entornoActual.tabla.get(nombre);
            }
            entornoActual = entornoActual.padre;
        }
        return null;
    }

    actualizar(nombre, nuevoValor) {
        let entornoActual = this;
        while (entornoActual !== null) {
            if (entornoActual.tabla.has(nombre)) {
                const simbolo = entornoActual.tabla.get(nombre);
                simbolo.valor = nuevoValor;
                entornoActual.tabla.set(nombre, simbolo);
                return true;
            }
            entornoActual = entornoActual.padre;
        }
        return false;
    }

    existe(nombre) {
        return this.obtener(nombre) !== null;
    }

    setCapturadorSalida(callback) {
        this.capturarSalida = callback;
    }
}
```

```
class Simbolo {
  constructor(nombre, tipo, valor, constante = false) {
    this.nombre = nombre;
    this.tipo = tipo;
    this.valor = valor;
    this.constante = constante;

    // Propiedades para vectores
    this.esVector = false;
    this.dimensiones = 0;
    this.tamanoes = []; // Para almacenar tamaños de cada dimension
    this.tipoElemento = ''; // Tipo de los elementos del vector

    // NUEVAS PROPIEDADES PARA OBJETOS
    this.esObjeto = false;
    this.tributos = []; // Lista de atributos del objeto
    this.metodos = []; // Lista de metodos del objeto
    this.tipoObjeto = null; // Para instancias: tipo del objeto padre
  }

  // Metodo para configurar un simbolo como objeto
  configurarObjeto(tributos = [], metodos = []) {
    this.esObjeto = true;
    this.tributos = tributos;
    this.metodos = metodos;
    this.valor = { tipo: 'definicion_objeto' };
  }

  // Metodo para configurar un simbolo como instancia de objeto
  configurarInstancia(tipoObjeto, valores = {}) {
    this.esObjeto = true;
    this.tipoObjeto = tipoObjeto;
    this.valor = valores;
  }

  // Metodos para manejar atributos de objetos
  obtenerAtributo(nombreAtributo) {
    return this.tributos.find(attr => attr.name === nombreAtributo);
  }
}
```

```
obtenerMetodo(nombreMetodo) {
    return this.metodos.find(method => method.name === nombreMetodo);
}

agregarMetodo(metodo) {
    this.metodos.push(metodo);
}

establecerValorAtributo(nombreAtributo, valor) {
    if (this.valor && typeof this.valor === 'object') {
        this.valor[nombreAtributo] = valor;
    }
}

obtenerValorAtributo(nombreAtributo) {
    if (this.valor && typeof this.valor === 'object') {
        return this.valor[nombreAtributo];
    }
    return null;
}

// Metodo para configurar un simbolo como vector
configurarVector(dimensiones, tamanios, tipoElemento) {
    this.esVector = true;
    this.dimensiones = dimensiones;
    this.tamanios = tamanios;
    this.tipoElemento = tipoElemento;

    // Inicializar el vector con valores por defecto
    if (dimensiones === 1) {
        this.valor =
Array(tamanios[0]).fill(this.getValorPorDefecto(tipoElemento));
    } else if (dimensiones === 2) {
        this.valor = Array(tamanios[0]).fill().map(
            () =>
Array(tamanios[1]).fill(this.getValorPorDefecto(tipoElemento))
        );
    }
}
```

```
// Metodo para configurar vector con valores literales
configurarVectorConValores(valores, tipoElemento) {
    this.esVector = true;
    this.tipoElemento = tipoElemento;

    if (Array.isArray(valores)) {
        if (Array.isArray(valores[0])) {
            // Vector 2D
            this.dimensiones = 2;
            this.tamanyos = [valores.length, valores[0].length];
            this.valor = valores;
        } else {
            // Vector 1D
            this.dimensiones = 1;
            this.tamanyos = [valores.length];
            this.valor = valores;
        }
    }
}

getValorPorDefecto(tipo) {
    switch (tipo) {
        case 'entero': return 0;
        case 'decimal': return 0.0;
        case 'booleano': return false;
        case 'caracter': return '\0';
        case 'cadena': return '';
        default: return null;
    }
}

// Obtener elemento del vector
obtenerElemento(indices) {
    if (!this.esVector) {
        throw new Error(`${this.nombre} no es un vector`);
    }

    if (indices.length !== this.dimensiones) {
```

```
        throw new Error(`Numero de indices (${indices.length}) no coincide con dimensiones (${this.dimensiones})`);
    }

    let elemento = this.valor;
    for (let i = 0; i < indices.length; i++) {
        const indice = indices[i];
        if (indice < 0 || indice >= this.tamanyos[i]) {
            throw new Error(`Indice ${indice} fuera de rango para dimension ${i + 1}`);
        }
        elemento = elemento[indice];
    }

    return elemento;
}

// Asignar elemento del vector
asignarElemento(indices, valor) {
    if (!this.esVector) {
        throw new Error(`${this.nombre} no es un vector`);
    }

    if (indices.length !== this.dimensiones) {
        throw new Error(`Numero de indices (${indices.length}) no coincide con dimensiones (${this.dimensiones})`);
    }

    let contenedor = this.valor;
    for (let i = 0; i < indices.length - 1; i++) {
        const indice = indices[i];
        if (indice < 0 || indice >= this.tamanyos[i]) {
            throw new Error(`Indice ${indice} fuera de rango para dimension ${i + 1}`);
        }
        contenedor = contenedor[indice];
    }

    const ultimoIndice = indices[indices.length - 1];
```

```
        if (ultimoIndice < 0 || ultimoIndice >=
this.tamano[indices.length - 1]) {
            throw new Error(`Indice ${ultimoIndice} fuera de rango para
dimension ${indices.length}`);
        }

        // Verificar tipo del valor asignado
        const tipoValor = typeof valor;
        const tipoEsperado = this.tipoElemento;

        // Conversion basica de tipos
        if (tipoEsperado === 'entero' && tipoValor === 'number') {
            valor = Math.trunc(valor);
        } else if (tipoEsperado === 'decimal' && tipoValor === 'number') {
            // Ya es decimal
        } else if (tipoEsperado === 'cadena' && tipoValor !== 'string') {
            valor = String(valor);
        } else if (tipoEsperado === 'booleano' && tipoValor !== 'boolean')
{
            valor = Boolean(valor);
        }

        contenedor[ultimoIndice] = valor;
        return true;
    }
}

// NUEVA CLASE: SymbolTable para manejar tablas de simbolos extendidas
class SymbolTable {
    constructor() {
        this.symbols = new Map();
    }

    add(symbol) {
        this.symbols.set(symbol.nombre, symbol);
    }

    get(nombre) {
        return this.symbols.get(nombre);
    }
}
```

```
has(nombre) {
    return this.symbols.has(nombre);
}

getAll() {
    return Array.from(this.symbols.values());
}
}

module.exports = { Entorno, Simbolo, SymbolTable };
```

[interprete.js](#)

```
// backend\src\analizadorSemantico\interprete.js
const { Entorno, Simbolo } = require('./entorno');

class Interprete {
    constructor() {
        this.entornoGlobal = new Entorno();
    }

    interpretar(ast, capturadorSalida = null) {
        try {
            console.log(" Ejecutando AST...");

            if (capturadorSalida) {
                this.entornoGlobal.setCapturadorSalida(capturadorSalida);
            }

            // Verificar que el AST tenga el metodo evaluar
            if (typeof ast.evaluar !== 'function') {
                throw new Error("El AST no tiene metodo evaluar");
            }

            // Ejecutar el programa
            ast.evaluar(this.entornoGlobal);
        }
    }
}
```

```
        console.log(" Ejecucion completada exitosamente");
    } catch (error) {
        console.error(` Error durante la ejecucion:
${error.message}`);
        throw error;
    }
}

limpiar() {
    this.entornoGlobal = new Entorno();
}
}

module.exports = Interprete;
```

[analyzerController.js](#)

```
const parser = require('../gramatica/parser');
const AnalizadorSemantico =
require('../analizadorSemantico/analizador_semantico');
const GraphvizGenerator = require('../graphviz/graphviz_generator'); //

const graphvizGenerator = new GraphvizGenerator(); //

const analyzeCode = (req, res) => {
    try {
        const { code } = req.body;

        if (!code || typeof code !== 'string') {
            return res.status(400).json({
                success: false,
                message: 'El codigo es requerido y debe ser una cadena de texto'
            });
        }

        console.log('Codigo recibido para analisis:', code);

        // Array para almacenar los tokens encontrados
```

```
const tokensEncontrados = [];  
const errors = [];  
let salidaConsola = "";  
  
try {  
  console.log('INICIANDO ANALISIS LEXICO Y SINTACTICO');  
  
  // Metodo 1: Intentar el enfoque de tu prueba original  
  let ast;  
  
  // Verificar si el parser tiene una clase Parser  
  if (parser.Parser) {  
    console.log('Usando parser con clase Parser');  
    const parserInstance = new parser.Parser();  
  
    // Intentar capturar tokens del lexer  
    if (parserInstance.lexer) {  
      const originalPerformAction =  
parserInstance.lexer.performAction;  
      parserInstance.lexer.performAction = function(yy, yy_,  
$avoiding_name_collisions, YY_START) {  
        const token = originalPerformAction.call(this, yy, yy_,  
$avoiding_name_collisions, YY_START);  
  
        if (yy_.yytext && yy_.yytext.trim() !== '') {  
          const tokenInfo = {  
            no: tokensEncontrados.length + 1,  
            token: yy_.yytext,  
            tipo: determinarTipoToken(yy_.yytext),  
            valor: yy_.yytext,  
            linea: yy_.yylineno || 1,  
            columna: yy_.yyloc ? yy_.yyloc.first_column : 0  
          };  
          tokensEncontrados.push(tokenInfo);  
          console.log(` TOKEN: "${tokenInfo.token}"  
(${tokenInfo.tipo}) - Linea ${tokenInfo.linea}, Col  
${tokenInfo.columna}`);  
        }  
  
        return token;  
      };  
    }  
  }  
}
```

```
    };  
  }  
  
  ast = parserInstance.parse(code);  
  
  } else {  
    // Metodo 2: Parser directo (como en tu respuesta actual)  
    console.log('Usando parser directo');  
    ast = parser.parse(code);  
  
    // Para este caso, intentemos extraer tokens del AST o generar una  
    lista basica  
    generarTokensDesdeCodigo(code, tokensEncontrados);  
  }  
  
  console.log('Analisis lexico y sintactico completado');  
  console.log('AST generado:', JSON.stringify(ast, null, 2));  
  
  // EJECUTAR ANALISIS SEMANTICO  
  console.log('INICIANDO ANALISIS SEMANTICO Y EJECUCION');  
  let resultadoSemantico = null;  
  
  try {  
    const analizadorSemantico = new AnalizadorSemantico();  
    resultadoSemantico = analizadorSemantico.analizar(ast);  
  
    if (resultadoSemantico.exito) {  
      console.log('Analisis semantico completado exitosamente');  
      salidaConsola = resultadoSemantico.salida || "Ejecucion  
completada sin salida";  
    } else {  
      console.log('Error en analisis semantico:',  
resultadoSemantico.error);  
      errors.push({  
        no: errors.length + 1,  
        tipoError: 'Semantico',  
        mensaje: resultadoSemantico.error,  
        linea: 1,  
        columna: 1  
      });  
    }  
  };
```

```
        salidaConsola = resultadoSemantico.salida || "Error durante la
ejecucion";
    }
} catch (errorSemantico) {
    console.log('Error durante analisis semantico:',
errorSemantico.message);
    errors.push({
        no: errors.length + 1,
        tipoError: 'Semantico',
        mensaje: errorSemantico.message,
        linea: 1,
        columna: 1
    });
    salidaConsola = `Error: ${errorSemantico.message}`;
}

// Estructura de respuesta exitosa
const response = {
    success: true,
    message: 'Analisis completado exitosamente',
    tokens: tokensEncontrados,
    errors: errors,
    ast: ast, // AST incluido en la respuesta
    consoleOutput: [
        'Analisis lexico completado',
        'Analisis sintactico completado',
        `Tokens procesados: ${tokensEncontrados.length}`,
        `Errores encontrados: ${errors.length}`,
        'AST generado correctamente',
        'Analisis semantico completado',
        '--- SALIDA DE CONSOLA ---',
        ...salidaConsola.split('\n')
    ],
    salidaConsola: salidaConsola
};

console.log('Respuesta enviada al frontend');
console.log('Salida de consola:', salidaConsola);
res.json(response);
```

```
} catch (parseError) {
  console.log('Error durante el analisis:', parseError.message);

  // Capturar informacion del error del parser
  if (parseError.hash) {
    const errorInfo = {
      no: errors.length + 1,
      tipoError: 'Sintactico',
      mensaje: parseError.message,
      linea: parseError.hash.line || parseError.hash.loc?.first_line
|| 1,
      columna: parseError.hash.loc?.first_column || 1,
      token: parseError.hash.token || 'Desconocido'
    };
    errors.push(errorInfo);

    console.log('Error detallado:', errorInfo);
  }

  // Respuesta con errores pero tokens encontrados hasta el momento
  const response = {
    success: false,
    message: 'Error en el analisis',
    tokens: tokensEncontrados,
    errors: errors,
    ast: null,
    consoleOutput: [
      `Tokens procesados: ${tokensEncontrados.length}`,
      `Error en analisis: ${parseError.message}`,
      `Errores encontrados: ${errors.length}`
    ],
    salidaConsola: `Error de sintaxis: ${parseError.message}`
  };

  res.json(response);
}

} catch (error) {
  console.error('Error general del servidor:', error);
}
```

```
res.status(500).json({
  success: false,
  message: 'Error interno del servidor',
  error: error.message,
  tokens: [],
  errors: [{
    no: 1,
    tipoError: 'Sistema',
    mensaje: `Error del servidor: ${error.message}`,
    linea: 1,
    columna: 1
  }],
  consoleOutput: [
    'X Error interno del servidor',
    'X No se pudo completar el analisis'
  ],
  salidaConsola: `Error del servidor: ${error.message}`
});
}
};

// Controlador para generar grafico del AST
const generateASTGraph = async (req, res) => {
  try {
    const { ast } = req.body;

    if (!ast) {
      return res.status(400).json({
        success: false,
        error: 'No se proporciono AST para generar el grafico'
      });
    }

    console.log(' Generando grafico del AST...');

    const result = await graphvizGenerator.generateASTImage(ast);

    console.log(' Grafico del AST generado exitosamente');
    res.json(result);
  }
}
```

```

    } catch (error) {
      console.error(' Error generando grafico AST:', error);
      res.status(500).json({
        success: false,
        error: error.message
      });
    }
  };

// Funcion para generar tokens basicos desde el codigo cuando no podemos
capturarlos del lexer
function generarTokensDesdeCodigo(codigo, tokensArray) {
  const lineas = codigo.split('\n');

  // Patrones para tokens compuestos
  const patronesCompuestos = [
    /con valor/g,
    /con metodo/g,
    /ingresar objeto/g,
    /de lo contrario/g
  ];

  lineas.forEach((linea, numeroLinea) => {
    let lineaProcesada = linea;

    // Primero reemplazar tokens compuestos con marcadores temporales
    patronesCompuestos.forEach((patron, index) => {
      lineaProcesada = lineaProcesada.replace(patron,
`@@COMPUESTO_${index}@@`);
    });

    // Dividir la linea en tokens basicos
    const tokensLinea =
lineaProcesada.split(/(\s+|;|\(|\)|\{|\}|=|,|\+|-|\*|\/|=|!|=|<|=|>|<|>|\.
|:|"|'|\`)/)
      .filter(token => token && token.trim() !== '');

    let columna = 1;
    tokensLinea.forEach((token) => {
      // Restaurar tokens compuestos

```

```

    let tokenFinal = token;
    if (token.startsWith('@@COMPUESTO_')) {
        const index = parseInt(token.replace('@@COMPUESTO_',
        '').replace('@@', ' '));
        const compuestos = ['con valor', 'con metodo', 'ingresar objeto',
        'de lo contrario'];
        tokenFinal = compuestos[index] || token;
    }

    const tokenInfo = {
        no: tokensArray.length + 1,
        token: tokenFinal.trim(),
        tipo: determinarTipoToken(tokenFinal.trim()),
        valor: tokenFinal.trim(),
        linea: numeroLinea + 1,
        columna: columna
    };

    tokensArray.push(tokenInfo);
    console.log(` TOKEN GENERADO: "${tokenInfo.token}"
    (${tokenInfo.tipo}) - Linea ${tokenInfo.linea}, Col
    ${tokenInfo.columna}`);

    columna += tokenFinal.length;
    });
    });
}

// Funcion auxiliar para determinar el tipo de token
function determinarTipoToken(token) {
    // Palabras reservadas basicas
    const palabrasReservadas = [
        'entero', 'decimal', 'booleano', 'caracter', 'cadena', 'vector',
        'si', 'o', 'de', 'lo', 'contrario', 'mientras', 'para', 'hacer',
        'hasta', 'que', 'detener', 'continuar', 'procedimiento', 'funcion',
        'retornar', 'ingresar objeto', 'objeto', 'con valor', 'con metodo',
        'imprimir', 'nl', 'ejecutar', 'tolower', 'toupper',
        'verdadero', 'falso', 'true', 'false', 'Verdadero', 'Falso'
    ];
};

```

```
// Tokens compuestos (deben verificarse primero)
const tokensCompuestos = [
  'con valor', 'con metodo', 'ingresar objeto', 'de lo contrario'
];

// Operadores y simbolos
const operadores = [
  '+', '-', '*', '/', '%', '^', '=', '==', '!=', '<', '>', '<=', '>=',
  '&&', '||', '!', '++', '--', '->'
];

// Delimitadores
const delimitadores = ['(', ')', '[', ']', '{', '}', ';', ',', '.', ':',
  '?'];

// Casts
const casts = [
  '(entero)', '(decimal)', '(caracter)', '(cadena)',
  '( entero )', '( decimal )', '( caracter )', '( cadena )'
];

// Primero verificar tokens compuestos (los mas especificos primero)
for (const compuesto of tokensCompuestos) {
  if (token.toLowerCase().includes(compuesto.toLowerCase())) {
    return 'Palabra Reservada';
  }
}

// Verificar casts
for (const cast of casts) {
  if (token.replace(/\s+/g, ' ').trim().toLowerCase() ===
    cast.replace(/\s+/g, ' ').trim().toLowerCase()) {
    return 'Operador Cast';
  }
}

// Verificar palabras reservadas individuales
if (palabrasReservadas.includes(token.toLowerCase())) {
  return 'Palabra Reservada';
}
```

```
// Verificar valores booleanos
else if (token.toLowerCase() === 'verdadero' || token.toLowerCase() ===
'falso' ||
        token.toLowerCase() === 'true' || token.toLowerCase() ===
'false') {
    return 'Valor Booleano';
}
// Verificar operadores
else if (operadores.includes(token)) {
    return 'Operador';
}
// Verificar delimitadores
else if (delimitadores.includes(token)) {
    return 'Delimitador';
}
// Verificar numeros
else if (/^-?\d+$/.test(token)) {
    return 'Numero Entero';
}
else if (/^-?\d+\.\d+$/.test(token)) {
    return 'Numero Decimal';
}
// Verificar cadenas de texto (entre comillas)
else if ((token.startsWith('"') && token.endsWith('"')) ||
        (token.startsWith("'") && token.endsWith("'"))) {
    return 'Cadena Texto';
}
// Verificar cadenas que contienen espacios y texto
else if (token.includes(' ') && /[a-zA-Z]/.test(token) &&
!/\d/.test(token)) {
    return 'Cadena Texto';
}
// Verificar identificadores
else if (/^[a-zA-Z_][a-zA-Z0-9_]*$/.test(token)) {
    return 'Identificador';
}
// Verificar comentarios
else if (token.startsWith('//') || token.startsWith('/*')) {
    return 'Comentario';
}
```

```
// Token no reconocido
else {
    return 'Desconocido';
}
}

module.exports = {
    analyzeCode,
    generateASTGraph // exportar la nueva funcion
};
```

graphviz_generator.js

```
const { exec } = require('child_process');
const fs = require('fs');
const path = require('path');

class GraphvizGenerator {
    constructor() {
        this.tempDir = path.join(__dirname, 'temp');
        this.ensureTempDir();
    }

    ensureTempDir() {
        if (!fs.existsSync(this.tempDir)) {
            fs.mkdirSync(this.tempDir, { recursive: true });
        }
    }

    generateDotFromAST(ast) {
        let dotCode = 'digraph AST {\n';
        dotCode += '    node [shape=box, style=filled, fillcolor=lightblue, fontname="Arial"];\n';
        dotCode += '    edge [fontname="Arial", fontsize=10];\n\n';
        dotCode += '    rankdir=TB;\n'; // Top to bottom layout
        dotCode += '    concentrate=true;\n\n';

        let nodeId = 0;
```

```

const nodeMap = new Map();

// Funcion recursiva para procesar nodos
const processNode = (node, parentId = null, edgeLabel = '') => {
  if (!node || typeof node !== 'object') return;

  const currentNodeId = nodeId++;

  // Crear etiqueta para el nodo
  let label = node.type || 'Unknown';
  if (node.name) label += `\\nNombre: ${node.name}`;
  if (node.value !== undefined) label += `\\nValor: ${node.value}`;
  if (node.tipo) label += `\\nTipo: ${node.tipo}`;
  if (node.operator) label += `\\nOperador: ${node.operator}`;
  if (node.kind) label += `\\nKind: ${node.kind}`;
  if (node.dataType) label += `\\nDataType: ${node.dataType}`;
  if (node.returnType) label += `\\nReturn: ${node.returnType}`;
  if (node.resultType) label += `\\nResult: ${node.resultType}`;

  // Limitar longitud de la etiqueta
  if (label.length > 100) {
    label = label.substring(0, 97) + '...';
  }

  dotCode += `  node${currentNodeId} [label="${label.replace(/"/g,
'\\\\"')}]";\n`;
  nodeMap.set(node, currentNodeId);

  // Conectar con el padre
  if (parentId !== null) {
    const edge = edgeLabel ? ` [label="${edgeLabel}"]` : '';
    dotCode += `  node${parentId} -> node${currentNodeId}${edge};\n`;
  }

  // Procesar hijos recursivamente segun la estructura del AST
  const processChild = (child, label = '') => {
    if (child && typeof child === 'object') {
      processNode(child, currentNodeId, label);
    }
  };
};

```

```

const processChildren = (children, label = '') => {
  if (Array.isArray(children)) {
    children.forEach((child, index) => {
      if (child && typeof child === 'object') {
        processNode(child, currentNodeId, `${label}[${index}]`);
      }
    });
  } else if (children && typeof children === 'object') {
    processNode(children, currentNodeId, label);
  }
};

// Procesar diferentes tipos de nodos
if (node.body) processChildren(node.body, 'body');
if (node.statements) processChildren(node.statements, 'statements');
if (node.condition) processChild(node.condition, 'condition');
if (node.thenBlock) processChild(node.thenBlock, 'then');
if (node.elseBlock) processChild(node.elseBlock, 'else');
if (node.elseIfs) processChildren(node.elseIfs, 'elseIf');
if (node.left) processChild(node.left, 'left');
if (node.right) processChild(node.right, 'right');
if (node.expr) processChild(node.expr, 'expr');
if (node.target) processChild(node.target, 'target');
if (node.value && typeof node.value === 'object')
processChild(node.value, 'value');
if (node.initialization) processChild(node.initialization, 'init');
if (node.update) processChild(node.update, 'update');
if (node.parameters) processChildren(node.parameters, 'param');
if (node.args) processChildren(node.args, 'arg');
if (node.valores) processChildren(node.valores, 'valor');
if (node.ids) {
  node.ids.forEach((id, index) => {
    dotCode += ` node${nodeId} [label="Identifier\\n${id}"];\n`;
    dotCode += ` node${currentNodeId} -> node${nodeId}
[label="ids[${index}]"];\n`;
    nodeId++;
  });
}
};

```

```
// Procesar el nodo raiz
if (ast.body && Array.isArray(ast.body)) {
  ast.body.forEach((node, index) => processNode(node, null,
`body[${index}]`));
} else {
  processNode(ast);
}

dotCode += '}\n';
return dotCode;
}

async generateASTImage(ast) {
  return new Promise((resolve, reject) => {
    try {
      // Generar codigo DOT
      const dotCode = this.generateDotFromAST(ast);

      // Crear archivos temporales
      const timestamp = Date.now();
      const dotFilePath = path.join(this.tempDir,
`ast_${timestamp}.dot`);
      const pngFilePath = path.join(this.tempDir,
`ast_${timestamp}.png`);

      // Escribir archivo DOT
      fs.writeFileSync(dotFilePath, dotCode);

      console.log(' Ejecutando Graphviz...');

      // Ejecutar Graphviz
      exec(`dot -Tpng "${dotFilePath}" -o "${pngFilePath}"`, (error,
stdout, stderr) => {
        if (error) {
          console.error(' Error ejecutando Graphviz:', error);
          console.error('stderr:', stderr);
          reject(new Error('Error al generar imagen con Graphviz: ' +
error.message));
        }
        return;
      });
    }
  });
}
```

```
    }

    // Leer la imagen generada
    if (fs.existsSync(pngFilePath)) {
        const imageBuffer = fs.readFileSync(pngFilePath);
        const base64Image = imageBuffer.toString('base64');
        const imageUrl = `data:image/png;base64,${base64Image}`;

        console.log(' Imagen generada exitosamente');

        // Limpiar archivos temporales
        this.cleanupTempFiles(dotFilePath, pngFilePath);

        resolve({
            success: true,
            imageUrl: imageUrl,
            dotCode: dotCode
        });
    } else {
        reject(new Error('No se pudo generar la imagen - archivo no encontrado'));
    }
});

} catch (error) {
    reject(error);
}
});
}

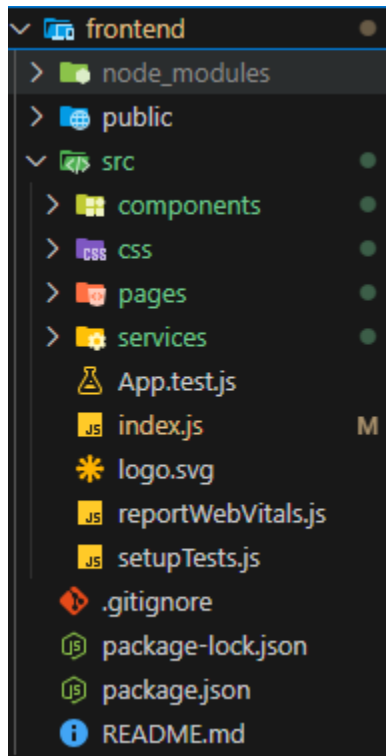
cleanupTempFiles(dotFilePath, pngFilePath) {
    try {
        if (fs.existsSync(dotFilePath)) {
            fs.unlinkSync(dotFilePath);
        }
        if (fs.existsSync(pngFilePath)) {
            fs.unlinkSync(pngFilePath);
        }
    } catch (error) {
        console.warn(' Error limpiando archivos temporales:', error);
    }
}
```

```
    }  
  }  
}  
  
module.exports = GraphvizGenerator;
```

[analyzer.js](#)

```
const express = require('express');  
const router = express.Router();  
const { analyzeCode, generateASTGraph } =  
  require('../controllers/analyzerController');  
  
// Ruta para analizar codigo  
router.post('/analyze', analyzeCode);  
router.post('/graph-ast', generateASTGraph);  
  
module.exports = router;
```

Frontend:



index.js

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './pages/App.jsx'
import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
import './css/index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

ConsoleOutput.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
```

```
import App from './pages/App.jsx'
import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
import './css/index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Editor.jsx

```
import React, { useRef, useEffect } from 'react';

function Editor({ id, editable }) {
  const editorRef = useRef(null);
  const className = editable ? 'text' : 'editor no-editable';

  // Contenido inicial para el editor de SimpliCode
  const getInitialContent = () => {
    if (id === 'editor') {
      return `// Escribe o sube un archivo de SimpliCode aqui`;
    }
    return '';
  };

  useEffect(() => {
    if (editorRef.current && id === 'editor') {
      editorRef.current.innerHTML = getInitialContent();
    }
  }, [id]);

  return (
    <div
      ref={editorRef}
      id={id}
      className={className}
      contentEditable={editable}
    >
```

```
        spellCheck="false"  
      />  
    );  
  }  
  
export default Editor;
```

Header.jsx

```
import React, { useRef } from 'react';  
  
function Header({ onGraphAST, astData, onNewFile, onOpenFile, onSaveFile,  
hasUnsavedChanges }) {  
  const fileInputRef = useRef(null);  
  
  const handleGraphClick = () => {  
    if (onGraphAST && astData) {  
      onGraphAST(astData);  
    } else {  
      alert('No hay AST disponible para graficar. Primero analiza algun  
codigo.');    }  
  };  
  
  const handleNewFile = () => {  
    if (hasUnsavedChanges) {  
      const confirm = window.confirm('Tienes cambios sin guardar. ¿Estas  
seguro de que quieres crear un nuevo archivo?');  
      if (!confirm) return;  
    }  
    onNewFile?.();  
  };  
  
  const handleOpenFile = () => {  
    if (hasUnsavedChanges) {  
      const confirm = window.confirm('Tienes cambios sin guardar. ¿Estas  
seguro de que quieres abrir otro archivo?');  
      if (!confirm) return;  
    }  
  }  
}
```

```
fileInputRef.current?.click();
};

const handleFileSelect = (event) => {
  const file = event.target.files[0];
  if (file) {
    onOpenFile?.(file);
  }
  // Limpiar el input para permitir seleccionar el mismo archivo otra
  vez
  event.target.value = '';
};

const handleSaveFile = () => {
  onSaveFile?.();
};

return (
  <nav className="navbar navbar-expand-lg navbar-dark">
    <div className="container-fluid">
      <a className="navbar-brand" href="#">
        SimpliCode IDE
      </a>
      <button
        className="navbar-toggler"
        type="button"
        data-bs-toggle="collapse"
        data-bs-target="#navbarNav"
        aria-controls="navbarNav"
        aria-expanded="false"
        aria-label="Toggle navigation"
      >
        <span className="navbar-toggler-icon"></span>
      </button>
      <div className="collapse navbar-collapse" id="navbarNav">
        <ul className="navbar-nav ms-auto">
          <li className="nav-item">
            <a className="nav-link active" href="#">Inicio</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
);
```

```

    { /* Botones de archivo en la barra principal */ }
    <li className="nav-item">
        <button
            className="btn btn-outline-light btn-sm ms-2"
            onClick={handleNewFile}
            title="Nuevo Archivo"
        >
            Nuevo
        </button>
    </li>
    <li className="nav-item">
        <button
            className="btn btn-outline-light btn-sm ms-2"
            onClick={handleOpenFile}
            title="Abrir Archivo"
        >
            Abrir
        </button>
    </li>
    <li className="nav-item">
        <button
            className="btn btn-outline-light btn-sm ms-2"
            onClick={handleSaveFile}
            disabled={!hasUnsavedChanges}
            title={hasUnsavedChanges ? "Guardar Archivo" : "Sin
cambios para guardar"}
        >
            {hasUnsavedChanges ? 'Guardar*' : 'Guardar'}
        </button>
    </li>
    <li className="nav-item">
        <button
            className="btn btn-outline-info btn-sm ms-2"
            onClick={handleGraphClick}
            disabled={!astData}
            title={astData ? "Graficar AST" : "No hay AST disponible"}
        >
            Graficar AST
        </button>
    </li>

```

```

    { /* Dropdown con mas opciones */ }
    <li className="nav-item dropdown">
      <a
        className="nav-link dropdown-toggle"
        href="#"
        id="navbarDropdown"
        role="button"
        data-bs-toggle="dropdown"
        aria-expanded="false"
      >
        Opciones
      </a>
      <ul className="dropdown-menu"
        aria-labelledby="navbarDropdown">
        <li>
          <button className="dropdown-item"
            onClick={handleNewFile}>
            Nuevo Archivo
          </button>
        </li>
        <li>
          <button className="dropdown-item"
            onClick={handleOpenFile}>
            Abrir Archivo
          </button>
        </li>
        <li>
          <button
            className="dropdown-item"
            onClick={handleSaveFile}
            disabled={!hasUnsavedChanges}
          >
            Guardar Archivo
          </button>
        </li>
        <li><hr className="dropdown-divider" /></li>
        <li>
          <button
            className="dropdown-item"

```

```

        onClick={handleGraphClick}
        disabled={!astData}
      >
        Graficar Arbol AST
      </button>
    </li>
  </ul>
</li>
</ul>
</div>

{/* Input oculto para seleccionar archivos */}
<input
  type="file"
  ref={fileInputRef}
  style={{ display: 'none' }}
  accept=".txt,.code,.scode,.simplicode"
  onChange={handleFileSelect}
/>
</div>
</nav>
);
}

export default Header;

```

Table.jsx

```

import React from 'react';

function Table({ id, type, data = [] }) {
  console.log(` Tabla ${type} recibio datos:`, data);

  const getHeaders = () => {
    switch (type) {
      case 'tokens':
        return ['No.', 'Token', 'Tipo', 'Valor', 'Linea', 'Columna'];
      case 'errors':

```

```
        return ['No.', 'Tipo Error', 'Mensaje', 'Linea', 'Columna'];
    default:
        return [];
    }
};

const headers = getHeaders();

const renderTableBody = () => {
    console.log(` Renderizando tabla ${type} con`, data.length,
'elementos');

    if (!data || data.length === 0) {
        return (
            <tr>
                <td colSpan={headers.length} className="text-center text-muted">
                    {type === 'tokens'
                        ? 'Los tokens apareceran aqui despues del analisis'
                        : 'No se encontraron errores'}
                </td>
            </tr>
        );
    }

    return data.map((item, index) => {
        console.log(` Procesando item ${index}:`, item);
        return (
            <tr key={index}>
                {headers.map((header, headerIndex) => {
                    const key = getKeyFromHeader(header);
                    const value = item[key] || '';
                    console.log(`   ${header} (${key}):`, value);
                    return <td key={headerIndex}>{value}</td>;
                })}
            </tr>
        );
    });
};

const getKeyFromHeader = (header) => {
```

```

const mapping = {
  'No.': 'no',
  'Token': 'token',
  'Tipo': 'tipo',
  'Valor': 'valor',
  'Linea': 'linea',
  'Columna': 'columna',
  'Tipo Error': 'tipoError',
  'Mensaje': 'mensaje'
};

return mapping[header] || header.toLowerCase();
};

return (
  <div>
    <table className="table table-striped table_responsive">
      <thead>
        <tr>
          {headers.map((header, index) => (
            <th key={index}>{header}</th>
          ))}
        </tr>
      </thead>
      <tbody className="table_body" id={id}>
        {renderTableBody()}
      </tbody>
    </table>
  </div>
);
}

export default Table;

```

App.jsx

```

import React, { useState, useEffect } from 'react';
import Header from '../components/Header';
import Editor from '../components/Editor';

```

```
import Table from '../components/Table';
import ConsoleOutput from '../components/ConsoleOutput';
import { analyzeCode, generateASTGraph } from '../services/api';
import '../css/App.css';

function App() {
  const [consoleOutput, setConsoleOutput] = useState('Aquí aparecerá los resultados de ejecución');
  const [analysisStatus, setAnalysisStatus] = useState(null);
  const [tokens, setTokens] = useState([]);
  const [errors, setErrors] = useState([]);
  const [astData, setAstData] = useState(null);
  const [graphImage, setGraphImage] = useState(null);

  // Estados para manejo de archivos
  const [currentFile, setCurrentFile] = useState(null);
  const [hasUnsavedChanges, setHasUnsavedChanges] = useState(false);
  const [editorContent, setEditorContent] = useState('');

  // Efecto para detectar cambios en el editor
  useEffect(() => {
    const checkForChanges = () => {
      const editor = document.getElementById('editor');
      if (editor) {
        const currentContent = editor.innerText || editor.textContent || '';

        if (currentContent !== editorContent) {
          setEditorContent(currentContent);
          if (currentFile && !hasUnsavedChanges) {
            setHasUnsavedChanges(true);
          }
        }
      }
    };

    // Verificar cambios periódicamente
    const interval = setInterval(checkForChanges, 1000);
    return () => clearInterval(interval);
  }, [currentFile, hasUnsavedChanges, editorContent]);
```

```
// Funcion para crear nuevo archivo
const handleNewFile = () => {
  setCurrentFile({
    name: 'nuevo_archivo.code',
    content: '',
    type: 'code'
  });
  setEditorContent('');
  setHasUnsavedChanges(false);
  setConsoleOutput('> Archivo nuevo creado\n> Listo para escribir
codigo...');

  // Limpiar el editor
  const editor = document.getElementById('editor');
  if (editor) {
    editor.innerHTML = '';
    editor.focus();
  }
};

// Funcion para abrir archivo
const handleOpenFile = async (file) => {
  try {
    const content = await readFileContent(file);
    setCurrentFile({
      name: file.name,
      content: content,
      type: getFileType(file.name)
    });
    setEditorContent(content);
    setHasUnsavedChanges(false);
    setConsoleOutput(`> Archivo "${file.name}" cargado exitosamente\n>
Listo para analizar...`);

    // Actualizar el editor
    const editor = document.getElementById('editor');
    if (editor) {
      editor.innerHTML = content;
    }
  } catch (error) {
```

```
        console.error('Error al abrir archivo:', error);
        setConsoleOutput(`> ERROR: No se pudo abrir el archivo
"${file.name}"\n> ${error.message}`);
    }
};

// Funcion para guardar archivo
const handleSaveFile = () => {
    if (!currentFile) {
        handleSaveAs();
        return;
    }

    const content = getEditorContent();
    downloadFile(content, currentFile.name, currentFile.type);
    setHasUnsavedChanges(false);
    setConsoleOutput(`> Archivo "${currentFile.name}" guardado
 exitosamente`);
};

// Funcion para guardar como
const handleSaveAs = () => {
    const content = getEditorContent();
    const fileName = prompt('Nombre del archivo:', currentFile?.name ||
'codigo.code');
    if (fileName) {
        const fileType = getFileType(fileName);
        downloadFile(content, fileName, fileType);
        setCurrentFile({
            name: fileName,
            content: content,
            type: fileType
        });
        setHasUnsavedChanges(false);
        setConsoleOutput(`> Archivo "${fileName}" guardado exitosamente`);
    }
};

// Funcion auxiliar para leer contenido de archivo
const readFileContent = (file) => {
```

```
    return new Promise((resolve, reject) => {
      const reader = new FileReader();
      reader.onload = (e) => resolve(e.target.result);
      reader.onerror = (e) => reject(new Error('No se pudo leer el
archivo'));
      reader.readAsText(file);
    });
  };

  // Funcion auxiliar para obtener tipo de archivo
  const getFileType = (fileName) => {
    const extension = fileName.split('.').pop()?.toLowerCase();
    return extension === 'txt' ? 'text/plain' : 'text/x-simplicode';
  };

  // Funcion auxiliar para obtener contenido del editor
  const getEditorContent = () => {
    const editor = document.getElementById('editor');
    return editor ? (editor.innerText || editor.textContent || '') : '';
  };

  // Funcion auxiliar para descargar archivo
  const downloadFile = (content, fileName, fileType) => {
    const blob = new Blob([content], { type: fileType });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = fileName;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);
  };

  // Funcion para manejar la grafica del AST
  const handleGraphAST = async (ast) => {
    try {
      console.log('Solicitando grafico del AST...');

      const result = await generateASTGraph(ast);
```

```

    if (result.success) {
      console.log('Grafico del AST generado exitosamente');
      setGraphImage(result.imageUrl);
      window.open(result.imageUrl, '_blank', 'width=1200,height=800');
    } else {
      console.error('Error al generar grafico:', result.error);
      alert('Error al generar el grafico: ' + result.error);
    }
  } catch (error) {
    console.error('Error de conexion:', error);
    alert('Error al conectar con el servidor para generar el grafico: '
+ error.message);
  }
};

const handleAnalyze = async () => {
  const content = getEditorContent();

  if (!content.trim()) {
    setConsoleOutput('> ERROR: El editor esta vacio\n> Por favor,
ingrese codigo SimpliCode para analizar');
    setAnalysisStatus('error');
    setTokens([]);
    setErrors([]);
    return;
  }

  setConsoleOutput('> Analizando codigo SimpliCode...\n> Conectando con
el backend...');
  setAnalysisStatus('processing');
  setTokens([]);
  setErrors([]);
  setAstData(null);

  try {
    console.log('Enviando codigo al backend:', content);
    const response = await analyzeCode(content);
    console.log('Respuesta del backend:', response);
  }

```

```
if (response.success) {
    setConsoleOutput(response.consoleOutput.join('\n'));
    setAnalysisStatus('success');

    // Actualizar tablas con datos del backend
    setTokens(response.tokens || []);
    setErrors(response.errors || []);

    // Guardar el AST si esta disponible
    if (response.ast) {
        console.log('AST recibido:', response.ast);
        setAstData(response.ast);
    } else {
        console.log('No se recibio AST en la respuesta');
    }

} else {
    setConsoleOutput(response.consoleOutput.join('\n'));
    setAnalysisStatus('error');

    // Mostrar tokens y errores incluso si hay errores
    setTokens(response.tokens || []);
    setErrors(response.errors || []);

    // Guardar AST incluso si hay errores
    if (response.ast) {
        console.log('AST recibido (con errores):', response.ast);
        setAstData(response.ast);
    }
}

} catch (error) {
    console.error(' Error al conectar con el backend:', error);
    setConsoleOutput(`> ✖ Error de conexion: ${error.message}\n>
Verifique que el backend este ejecutandose en http://localhost:3000`);
    setAnalysisStatus('error');
    setTokens([]);
    setErrors([]);
    setAstData(null);
}

};
```

```

return (
  <div>
    <Header
      onGraphAST={handleGraphAST}
      astData={astData}
      onNewFile={handleNewFile}
      onOpenFile={handleOpenFile}
      onSaveFile={handleSaveFile}
      hasUnsavedChanges={hasUnsavedChanges}
    />

    <div className="container_user">
      {/* Informacion del archivo actual */}
      {currentFile && (
        <div className="row mb-3">
          <div className="col-12">
            <div className={`alert ${hasUnsavedChanges ? 'alert-warning'
: 'alert-info'} py-2`} role="alert">
              <small>
                <strong>Archivo:</strong> {currentFile.name}
                {hasUnsavedChanges && <span className="badge bg-warning
ms-2">Modificado</span>}
              </small>
            </div>
          </div>
        </div>
      )}

      {/* Primera fila - Editores */}
      <div className="row justify-content-center mb-4">
        <div className="col-md-6 mb-3">
          <h5>
            Editor de Texto - SimpliCode
            {currentFile && <small className="text-muted
ms-2">({currentFile.name})</small>}
          </h5>
          <Editor id="editor" editable={true}
initialContent={editorContent} />
        </div>

```

```

    <div className="col-md-6 mb-3">
      <h5>Salida por Consola</h5>
      <ConsoleOutput content={consoleOutput} status={analysisStatus}
/>

    </div>
  </div>

  {/* Boton Analizar */}
  <div className="row justify-content-center mb-4">
    <div className="col-12 text-center">
      <button
        id="analyze"
        className="btn btn-primary btn_user"
        onClick={handleAnalyze}
        disabled={analysisStatus === 'processing'}
      >
        {analysisStatus === 'processing' ? 'Analizando...' :
'Analizar'}
      </button>

      {/* Boton Guardar Como */}
      <button
        className="btn btn-outline-secondary btn_user ms-2"
        onClick={handleSaveAs}
        title="Guardar como..."
      >
        Guardar Como
      </button>

      {analysisStatus === 'success' && (
        <div className="alert alert-success mt-2" role="alert">
          Analisis completado exitosamente
          {astData && ' - AST disponible para graficar'}
        </div>
      )}
      {analysisStatus === 'error' && (
        <div className="alert alert-danger mt-2" role="alert">
          ✖ Se encontraron errores en el analisis
          {astData && ' - AST parcial disponible para graficar'}
        </div>
      )}
    </div>
  </div>

```

```

    })
  </div>
</div>

{/* Segunda fila - Tablas */}
<div className="row">
  <div className="col-md-6 mb-3">
    <h5>Tabla de Tokens ({tokens.length})</h5>
    <Table id="tokens" type="tokens" data={tokens} />
  </div>
  <div className="col-md-6 mb-3">
    <h5>Errores ({errors.length})</h5>
    <Table id="errors" type="errors" data={errors} />
  </div>
</div>
</div>

{/* Modal para mostrar el grafico */}
{graphImage && (
  <div className="modal fade show d-block" style={{backgroundColor:
'rgba(0,0,0,0.5)'}}>
    <div className="modal-dialog modal-xl">
      <div className="modal-content">
        <div className="modal-header">
          <h5 className="modal-title">Arbol de Sintaxis Abstracta
(AST)</h5>
          <button
            type="button"
            className="btn-close"
            onClick={() => setGraphImage(null)}
          ></button>
        </div>
        <div className="modal-body text-center">
          <img
            src={graphImage}
            alt="Arbol AST"
            style={{maxWidth: '100%', height: 'auto', border: '1px
solid #ddd'}}
          />
        </div>
      </div>
    </div>
  )}

```

```
        <div className="modal-footer">
          <button
            type="button"
            className="btn btn-secondary"
            onClick={() => setGraphImage(null)}
          >
            Cerrar
          </button>
          <button
            type="button"
            className="btn btn-primary"
            onClick={() => window.open(graphImage, '_blank')}
          >
            Abrir en nueva pestaña
          </button>
        </div>
      </div>
    </div>
  </div>
  )}
</div>
);
}

export default App;
```